

Sesión 1

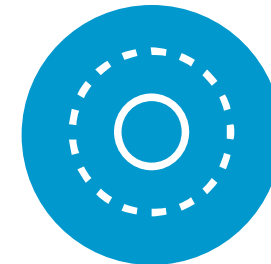
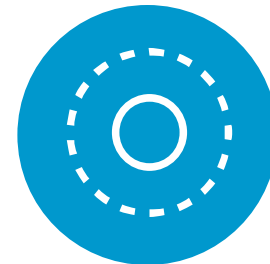
29 Abril 2025

- 1. Sistemas Big data**
- 2. Apache Kafka**
- 3. BBDD NoSQL**
- 4. Redshift**

1/ Sistemas Big Data

UF1 Aplicación de técnicas de integración, procesamiento y análisis de información.

L02. Técnicas y procesos de extracción de la información de los datos .

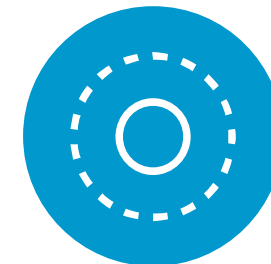
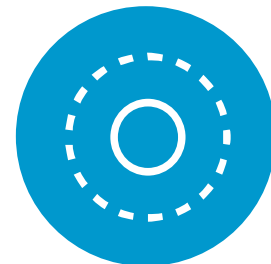


UF2 Técnicas de representación y minería de datos

L03. Métodos y algoritmos de minería de datos

UF 3 Sistemas de almacenamiento

Sistemas de gestión almacenamiento HDFS.
Apache Kafka, Motores y bbdd NoSQL



UF 4 Conceptos y visualizacion

L04 Herramientas de visualización de datos

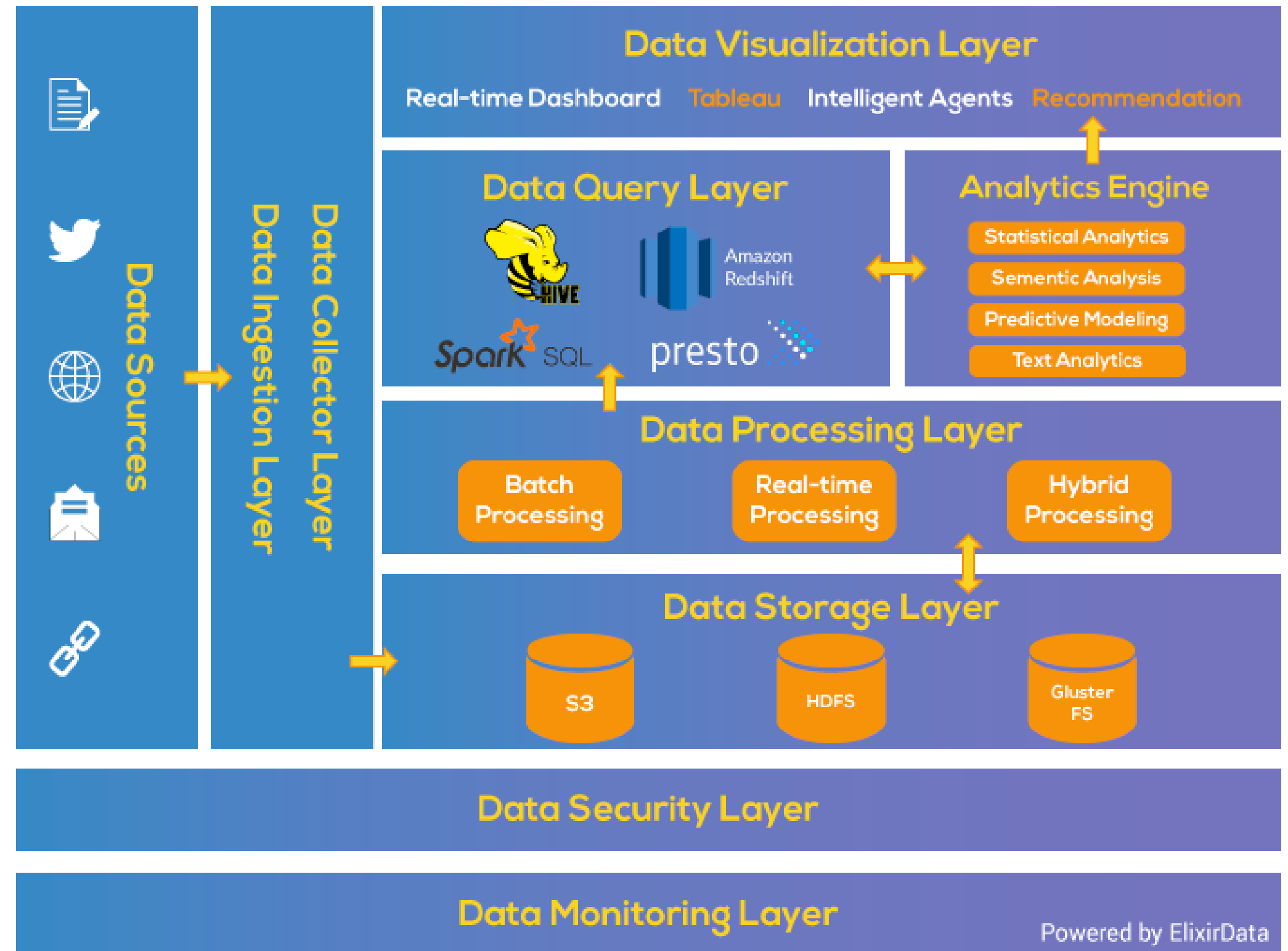
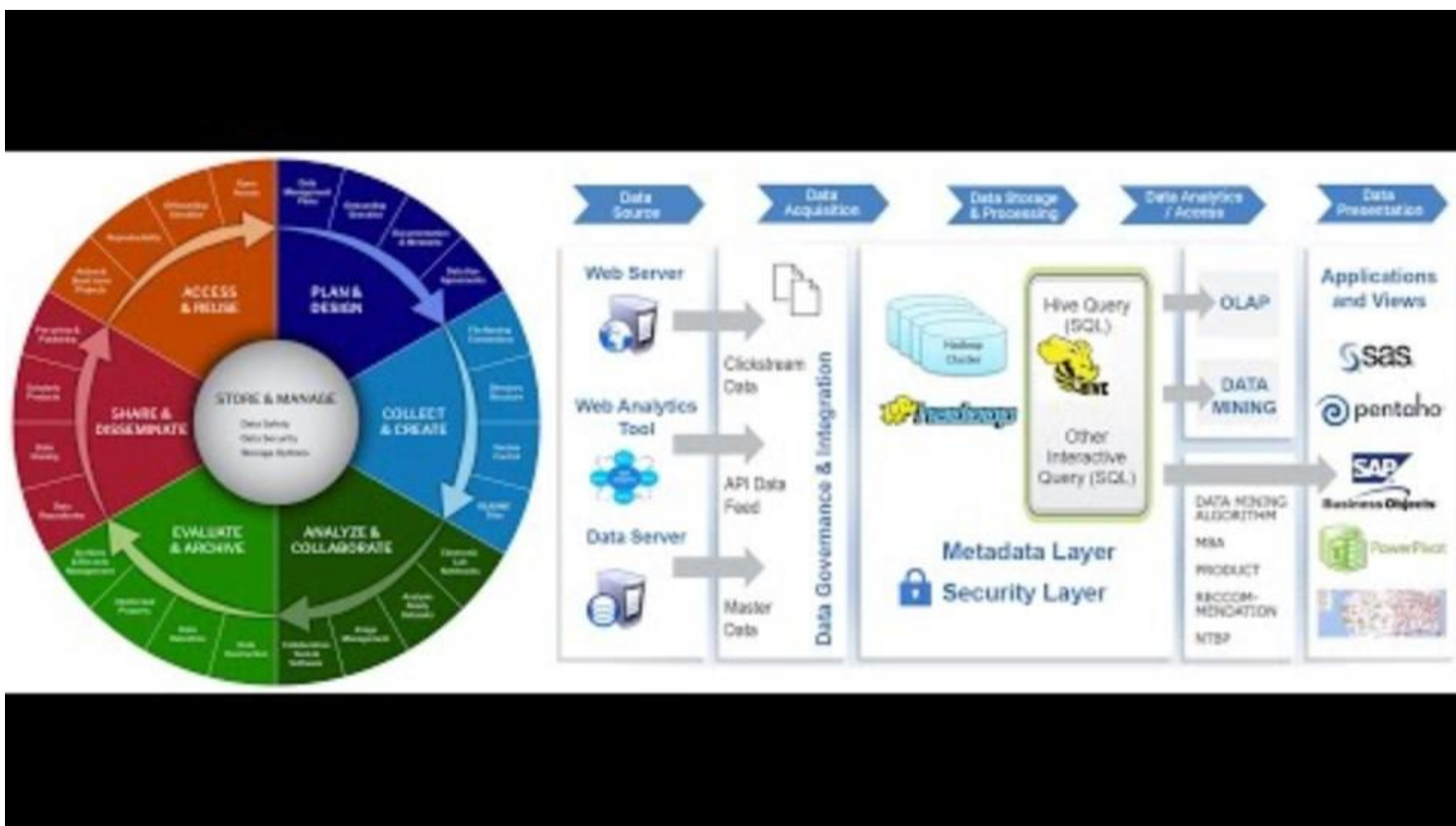
El ciclo de vida del dato y la arquitectura Big Data están íntimamente relacionados, ya que ambos describen el recorrido que siguen los datos, desde su creación hasta su eliminación o transformación en conocimiento útil. Cada fase del ciclo de vida del dato está respaldada por una o varias capas de la arquitectura Big Data, facilitando la administración y el procesamiento a gran escala.



Arquitectura Big Data: Definición y Componentes

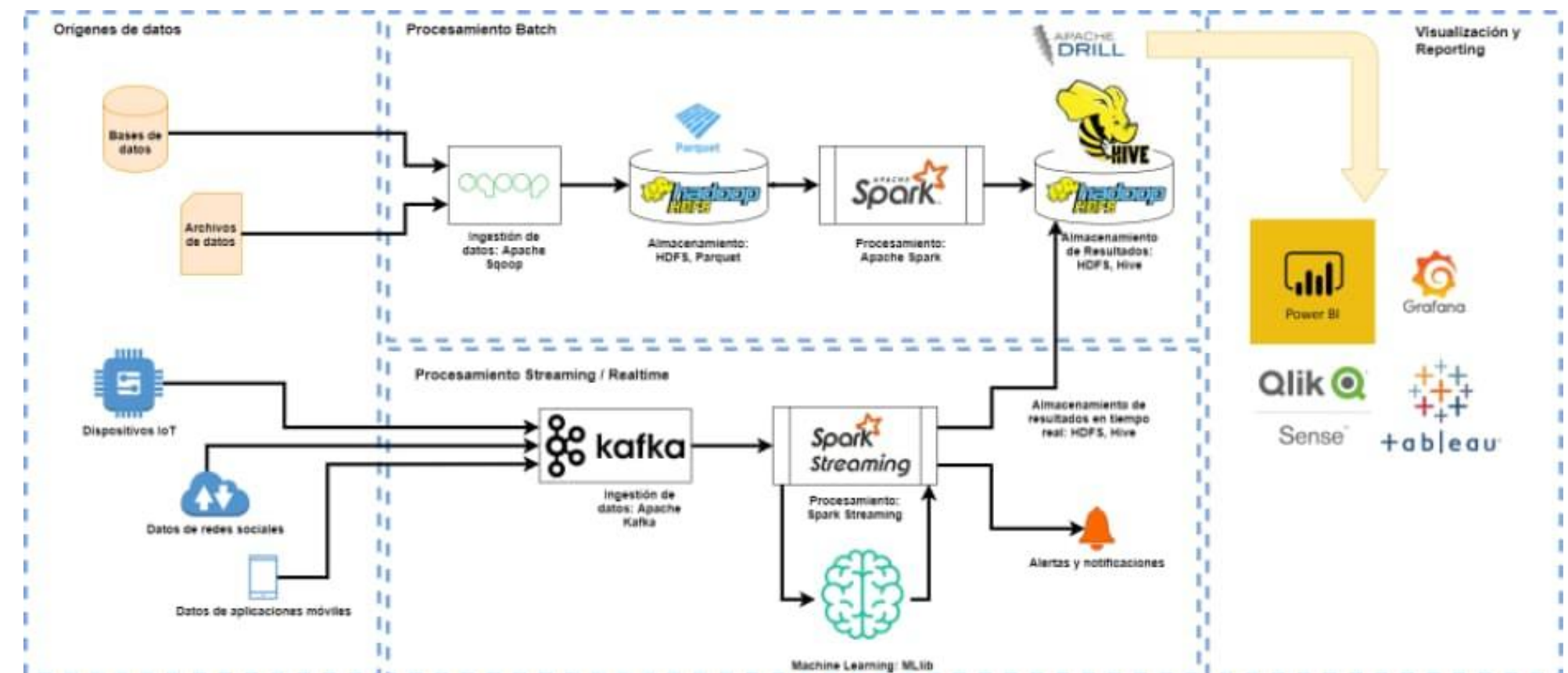
Una arquitectura Big Data es el diseño conceptual y la estructura que define cómo se recopilan, almacenan, procesan y analizan grandes volúmenes de datos en una organización.

Está compuesta por varias **capas** y **componentes**, cada una con una función específica, trabajando juntas para manejar la complejidad del procesamiento de datos masivos y heterogéneos



Hadoop es un framework de software de código abierto que permite almacenar y procesar grandes conjuntos de datos de forma distribuida y paralela en clústeres de computadoras. Es una herramienta fundamental en el mundo del Big Data debido a su escalabilidad y capacidad para manejar datos estructurados y no estructurados.

- **Escalabilidad:** Capacidad de manejar grandes volúmenes de datos.
- **Costo-efectividad:** Utiliza hardware commodity para construir clústeres.
- **Tolerancia a fallos:** Replicación de datos para garantizar la disponibilidad.
- **Flexibilidad:** Soporta una amplia variedad de formatos de datos.
- **Comunidad activa:** Gran comunidad de usuarios y desarrolladores.
- **Casos de Uso Típicos:**
 - **Análisis de grandes volúmenes de datos:** Log files, datos de sensores, redes sociales, etc.
 - **Almacenamiento de datos históricos:** Archivos de respaldo, registros de transacciones.
 - **Procesamiento de datos en tiempo real:** Análisis de streaming de datos.
 - **Machine Learning:** Entrenamiento de modelos de machine learning en grandes conjuntos de datos.



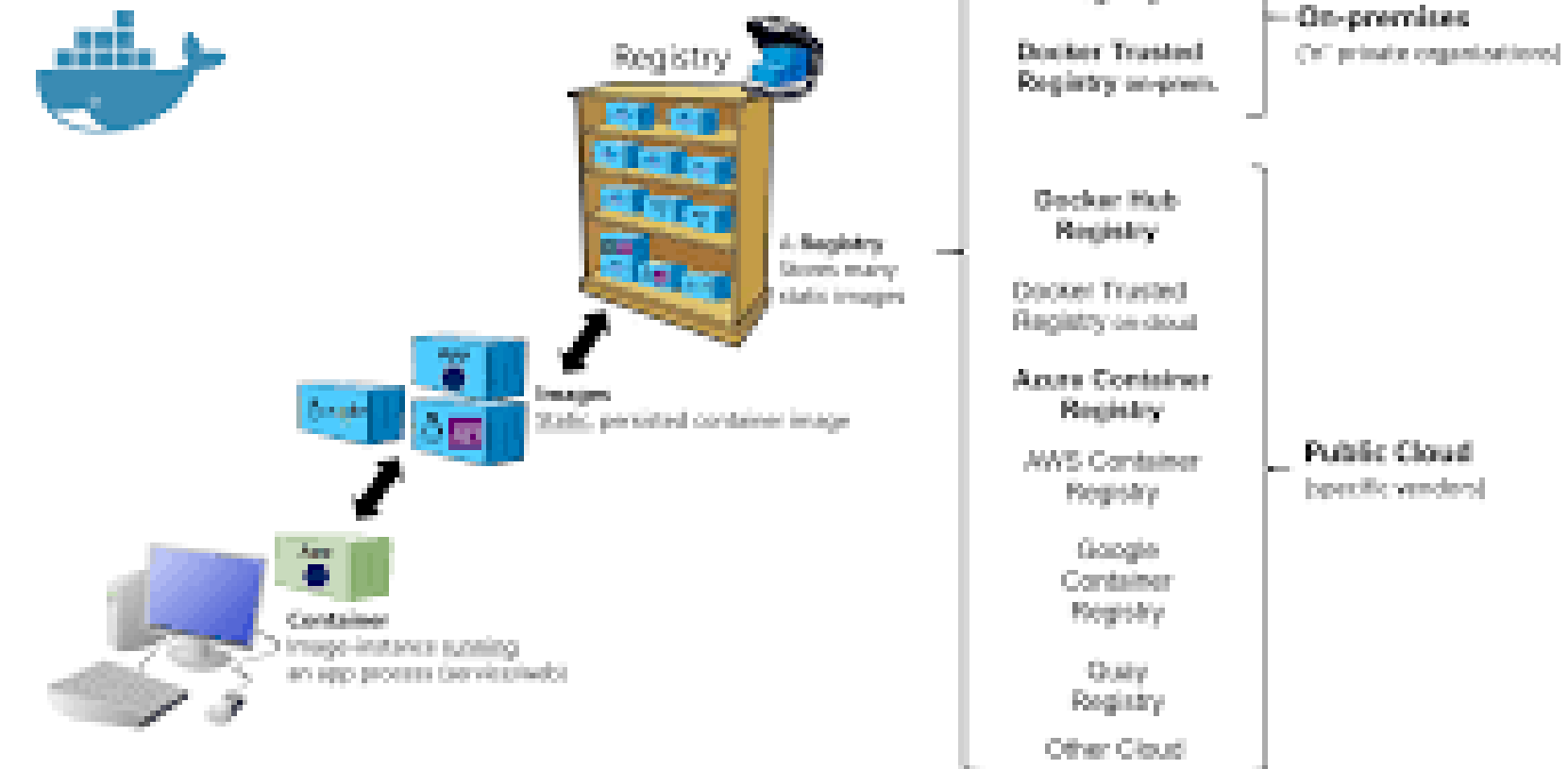
Imagina un contenedor como una caja perfectamente empaquetada.

- **Dentro de la caja:** Tienes todo lo necesario para que una aplicación funcione: el código de la aplicación, las bibliotecas que necesita, las herramientas del sistema, la configuración, etc.
- **La caja es independiente:** No importa dónde coloques la caja (en tu ordenador, en un servidor en la nube, etc.), la aplicación que está dentro siempre tendrá todo lo que necesita para funcionar correctamente, sin depender demasiado del sistema operativo subyacente.
- **La caja es ligera:** Comparada con una máquina virtual completa, la caja (el contenedor) comparte el núcleo del sistema operativo anfitrión, lo que la hace mucho más rápida de iniciar y consumir menos recursos.

En resumen, un contenedor es un entorno aislado y portátil que contiene todo lo necesario para ejecutar una aplicación. Esto asegura que la aplicación funcione de manera consistente en diferentes entornos.

- Un **contenedor** es un entorno aislado que empaqueta una aplicación y sus dependencias para asegurar su portabilidad y consistencia.
- **Docker** es la plataforma líder que proporciona las herramientas para construir, distribuir y ejecutar estos contenedores de manera eficiente.

Basic taxonomy in Docker

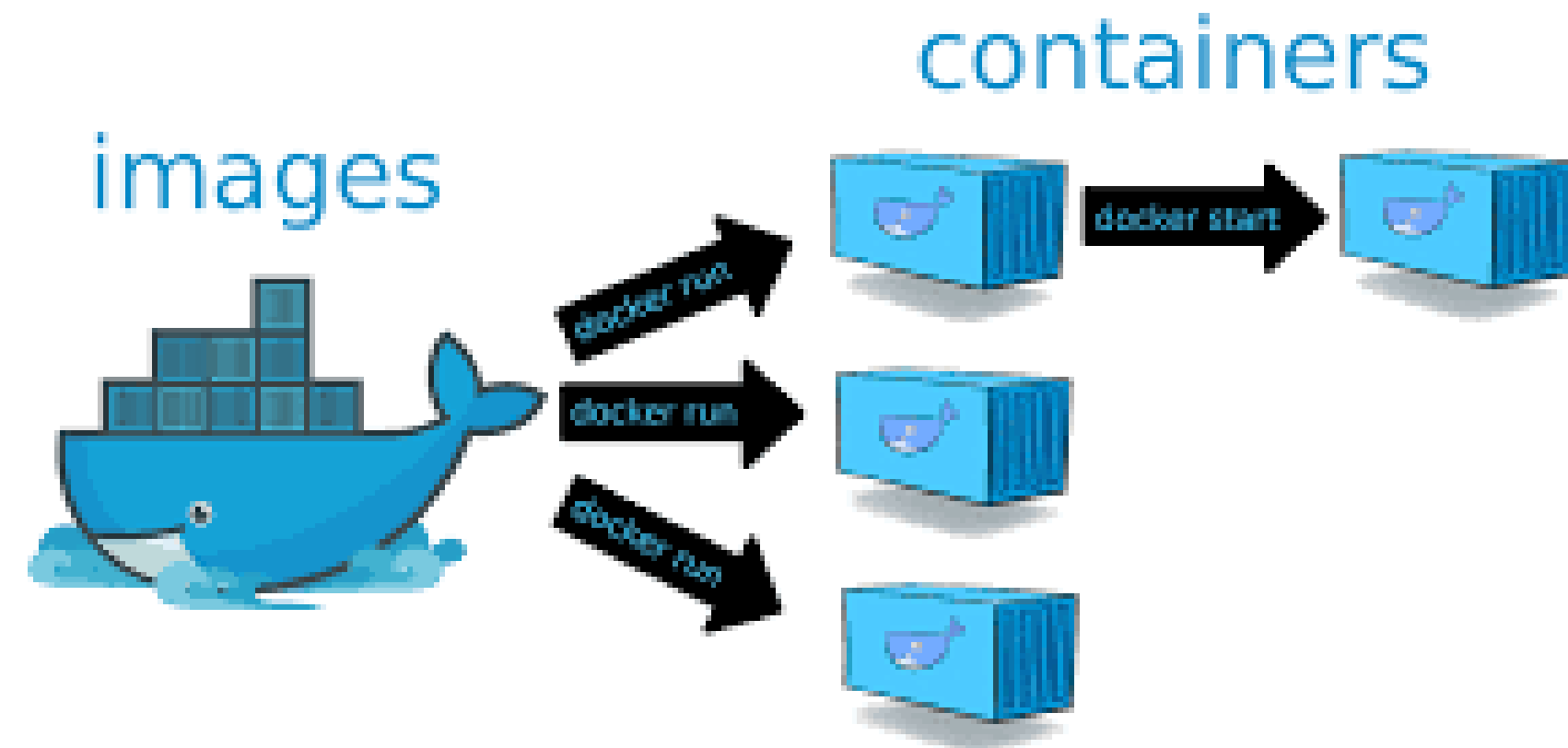


¿qué es Docker?

Docker es una plataforma que facilita la creación, gestión y ejecución de estos contenedores. Piénsalo como la herramienta que te permite:

- **Construir las cajas (contenedores):** Docker proporciona herramientas para definir cómo se debe empaquetar una aplicación y sus dependencias en una "imagen" (que sería como el plano de la caja).
- **Distribuir las cajas:** Docker Hub es un registro público (como un almacén) donde puedes encontrar y compartir imágenes de contenedores preconstruidas.
- **Ejecutar las cajas:** Docker Engine es el motor que realmente ejecuta los contenedores en tu sistema. Puedes iniciar, detener, mover y gestionar los contenedores de manera sencilla.

Piensa en Docker como el conjunto de herramientas y la infraestructura que hace que el uso de contenedores sea práctico y eficiente.



La relación entre Docker/contenedores y la Inteligencia Artificial (IA) es **fundamental y altamente beneficiosa** en el desarrollo, despliegue y gestión de aplicaciones de IA. Permíteme explicarte por qué:

1. Entornos Consistentes y Reproducibles:

- **El problema en IA:** El desarrollo de modelos de IA a menudo depende de bibliotecas específicas (TensorFlow, PyTorch, scikit-learn), versiones particulares de Python y otras dependencias del sistema. Configurar manualmente estos entornos puede ser tedioso, propenso a errores y difícil de replicar en diferentes máquinas o servidores.
- **La solución con contenedores:** Docker permite empaquetar todo el entorno necesario para una aplicación de IA (código, bibliotecas, dependencias, sistema operativo base) en una imagen. Esta imagen garantiza que la aplicación se ejecutará de manera consistente, sin importar el entorno subyacente. Esto es crucial para la reproducibilidad de los experimentos de IA y para asegurar que un modelo entrenado en un entorno funcione igual en producción.

2. Aislamiento y Gestión de Dependencias:

- **El problema en IA:** Diferentes proyectos de IA pueden requerir diferentes versiones de las mismas bibliotecas. Gestionar estas dependencias a nivel del sistema puede generar conflictos y problemas de compatibilidad.
- **La solución con contenedores:** Cada contenedor aísla su propio conjunto de dependencias. Esto significa que puedes tener múltiples proyectos de IA ejecutándose en la misma máquina, cada uno con sus propias bibliotecas y versiones, sin que interfieran entre sí. Docker facilita la gestión de estas dependencias dentro de cada contenedor.

3. Despliegue Sencillo y Portátil:

- **El problema en IA:** Desplegar modelos de IA entrenados en diferentes entornos (local, nube, edge devices) puede ser complejo debido a las diferencias en la infraestructura.
- **La solución con contenedores:** Una vez que una aplicación de IA está empaquetada en un contenedor Docker, se puede ejecutar en cualquier sistema que tenga Docker instalado. Esto simplifica enormemente el proceso de despliegue en diversas plataformas, desde servidores en la nube hasta dispositivos IoT. La portabilidad de los contenedores agiliza la transición del desarrollo a la producción.

4. Escalabilidad y Orquestación:

- **El problema en IA:** Las aplicaciones de IA, especialmente las que realizan inferencia en tiempo real, a menudo requieren escalabilidad para manejar picos de demanda. Gestionar múltiples instancias de una aplicación de IA puede ser complicado.
- **La solución con contenedores:** Docker se integra perfectamente con herramientas de orquestación de contenedores como Kubernetes y Docker Swarm. Estas herramientas permiten gestionar y escalar automáticamente múltiples contenedores de una aplicación de IA, distribuyendo la carga y asegurando la disponibilidad. Esto es esencial para construir sistemas de IA robustos y escalables.

5. Desarrollo Colaborativo:

- **El problema en IA:** En equipos de desarrollo de IA, asegurar que todos los miembros trabajen en el mismo entorno puede ser un desafío.
- **La solución con contenedores:** Docker permite definir un entorno de desarrollo estándar a través de un Dockerfile. Compartir este archivo garantiza que todos los miembros del equipo trabajen con las mismas herramientas y dependencias, reduciendo los problemas de "funciona en mi máquina".

HDFS vs. S3, Blob Storage, Google Cloud Storage

HDFS (Hadoop Distributed File System)

- **Pros:**
 - **Optimizado para el procesamiento de datos:** HDFS está diseñado para funcionar estrechamente con motores de procesamiento como MapReduce y Spark, lo que permite un procesamiento de datos de alto rendimiento.
 - **Localidad de los datos:** HDFS intenta almacenar los datos cerca de los nodos de computación para minimizar el movimiento de datos y mejorar el rendimiento del procesamiento.
 - **Tolerancia a fallos:** HDFS replica datos en múltiples nodos para garantizar la disponibilidad y la durabilidad.
 - **Adecuado para grandes volúmenes de datos:** HDFS puede manejar petabytes de datos y escalar a miles de nodos.
- **Contras:**
 - **Infraestructura local:** Requiere configurar y mantener tu propia infraestructura de hardware, lo que puede ser costoso y complejo.
 - **Mayor sobrecarga operativa:** Administrar un clúster de HDFS puede ser más exigente en términos de administración del sistema que usar un servicio en la nube.
 - **No es ideal para todos los casos de uso:** HDFS está optimizado para el procesamiento por lotes y puede no ser la mejor opción para casos de uso que requieren acceso de baja latencia o acceso aleatorio a los datos.

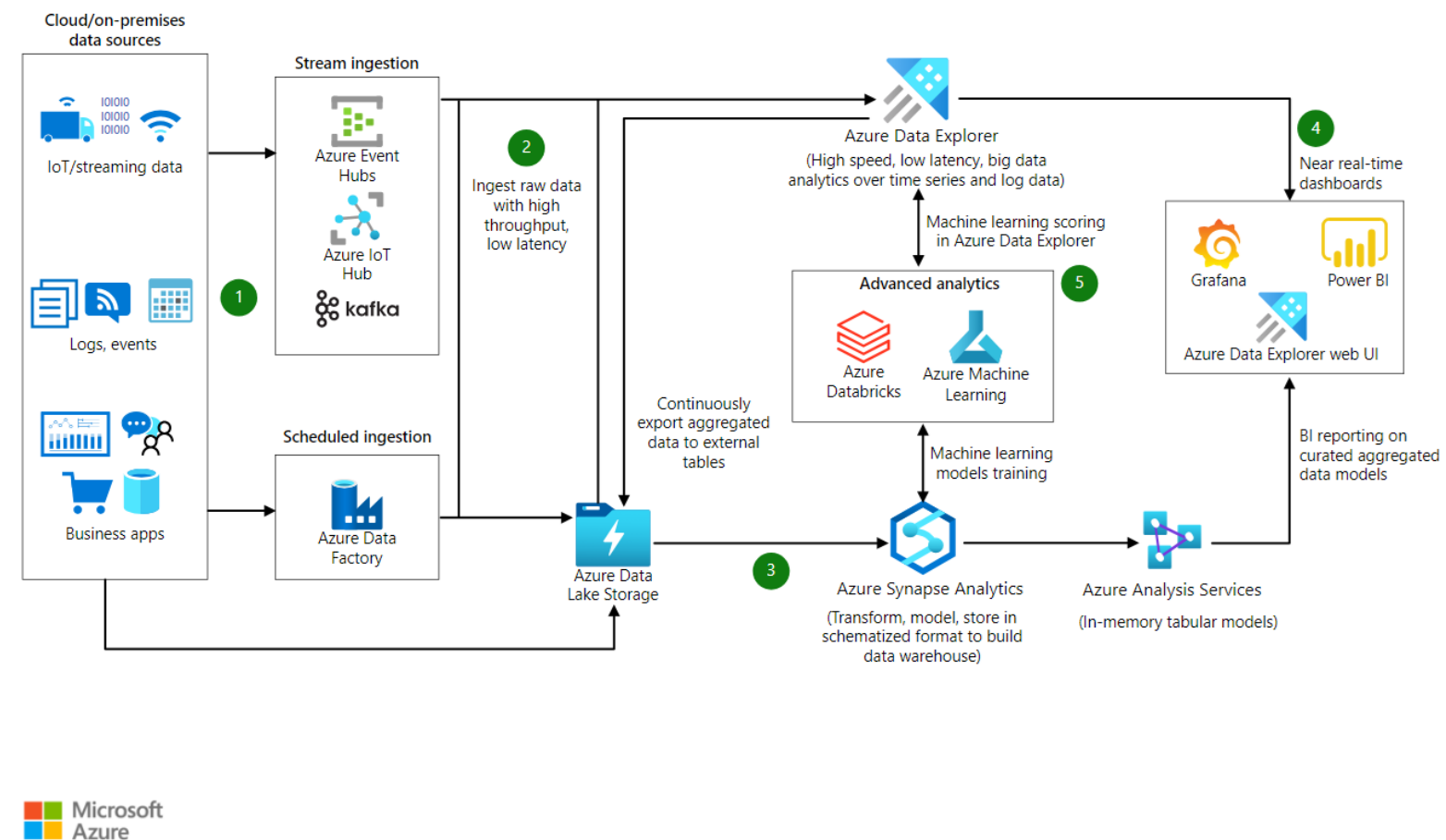
¿Cuándo elegir HDFS?

- **Procesamiento de datos a gran escala:** Si su caso de uso implica procesar grandes volúmenes de datos utilizando MapReduce, Spark u otros motores de procesamiento de datos, HDFS puede proporcionar un mejor rendimiento debido a la localidad de los datos.
- **Entornos locales:** Si tiene requisitos normativos o de seguridad específicos que le obligan a mantener los datos en las instalaciones, HDFS puede ser una buena opción.
- **Control sobre la infraestructura:** Si necesita un control total sobre la infraestructura de almacenamiento, HDFS le brinda la capacidad de personalizar y optimizar su hardware y configuración.

Azure proporciona una plataforma completa para gestionar y analizar grandes conjuntos de datos, desde la ingesta hasta la visualización. A diferencia de GCP, Azure ofrece una mayor flexibilidad al permitir la integración de tecnologías open source con servicios nativos..

Componentes Clave:

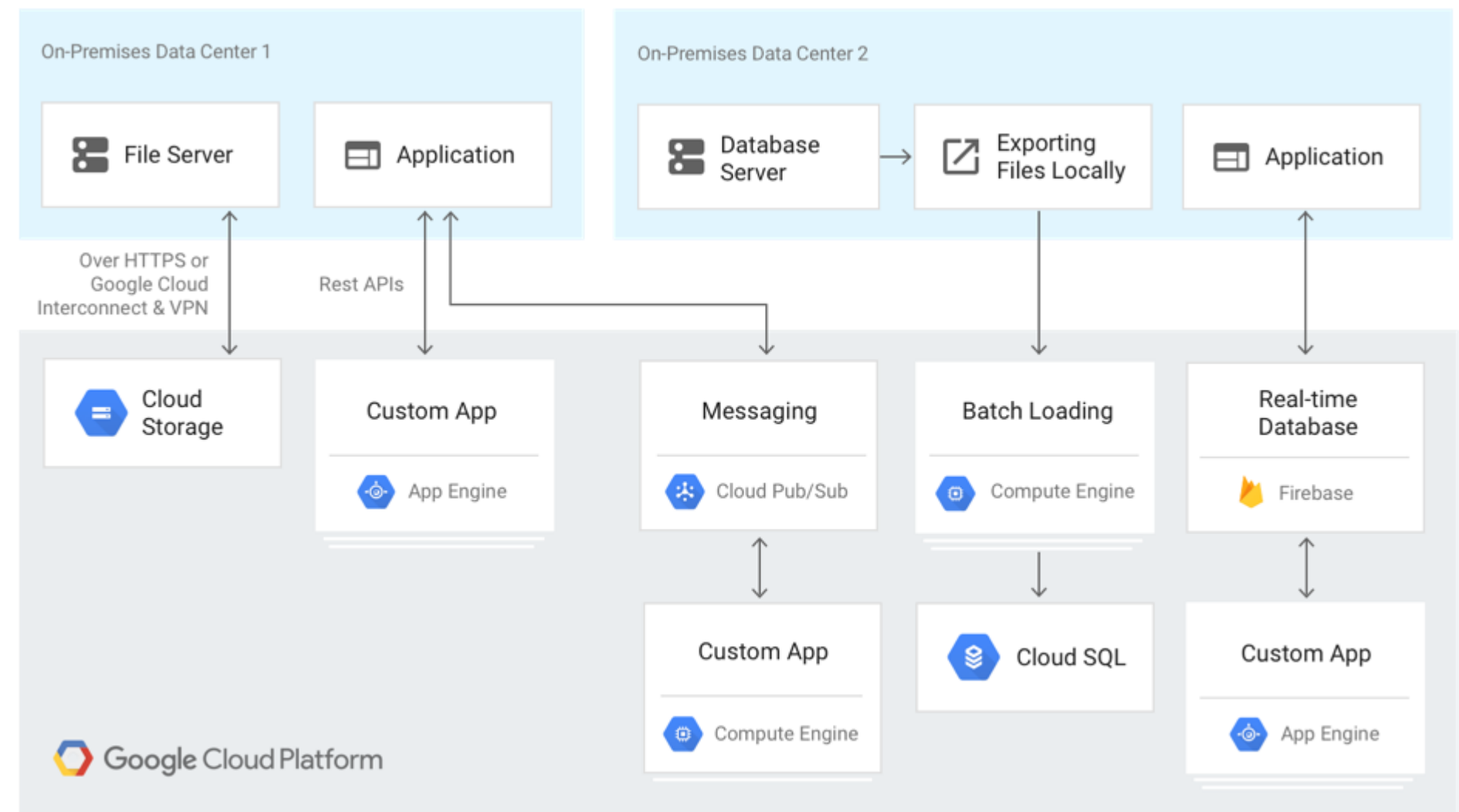
- **Azure Data Lake Storage Gen2:** Un sistema de archivos escalable y duradero para almacenar datos sin estructurados de cualquier tamaño.
- **Azure Synapse Analytics:** Un servicio unificado que combina un almacén d datos, un área de trabajo de Apache Spark y un servicio de integración de datos. Permite realizar consultas SQL, ejecutar trabajos de Spark y construir pipelines de ETL.
- **Azure HDInsight:** Un servicio administrado que permite ejecutar clústeres de Hadoop, Spark, Hive, HBase y Kafka en Azure. Ideal para cargas de trabajo de procesamiento de datos a gran escala.
- **Azure Stream Analytics:** Un servicio de procesamiento de streaming de datos en tiempo real basado en consultas SQL. Perfecto para aplicaciones de IoT y análisis de flujos de datos.
- **Azure Databricks:** Una plataforma de análisis basada en Apache Spark optimizada para Azure. Ofrece una experiencia de usuario colaborativa y un rendimiento excepcional.



Google Cloud Platform (GCP) es un conjunto de servicios en la nube que proporciona Google, y ofrece un amplio abanico de herramientas y soluciones para datos e Inteligencia Artificial (IA). GCP está diseñado para ayudar a las empresas a almacenar, procesar, analizar y obtener valor de sus datos, y para construir e implementar aplicaciones de IA.

Componentes Clave

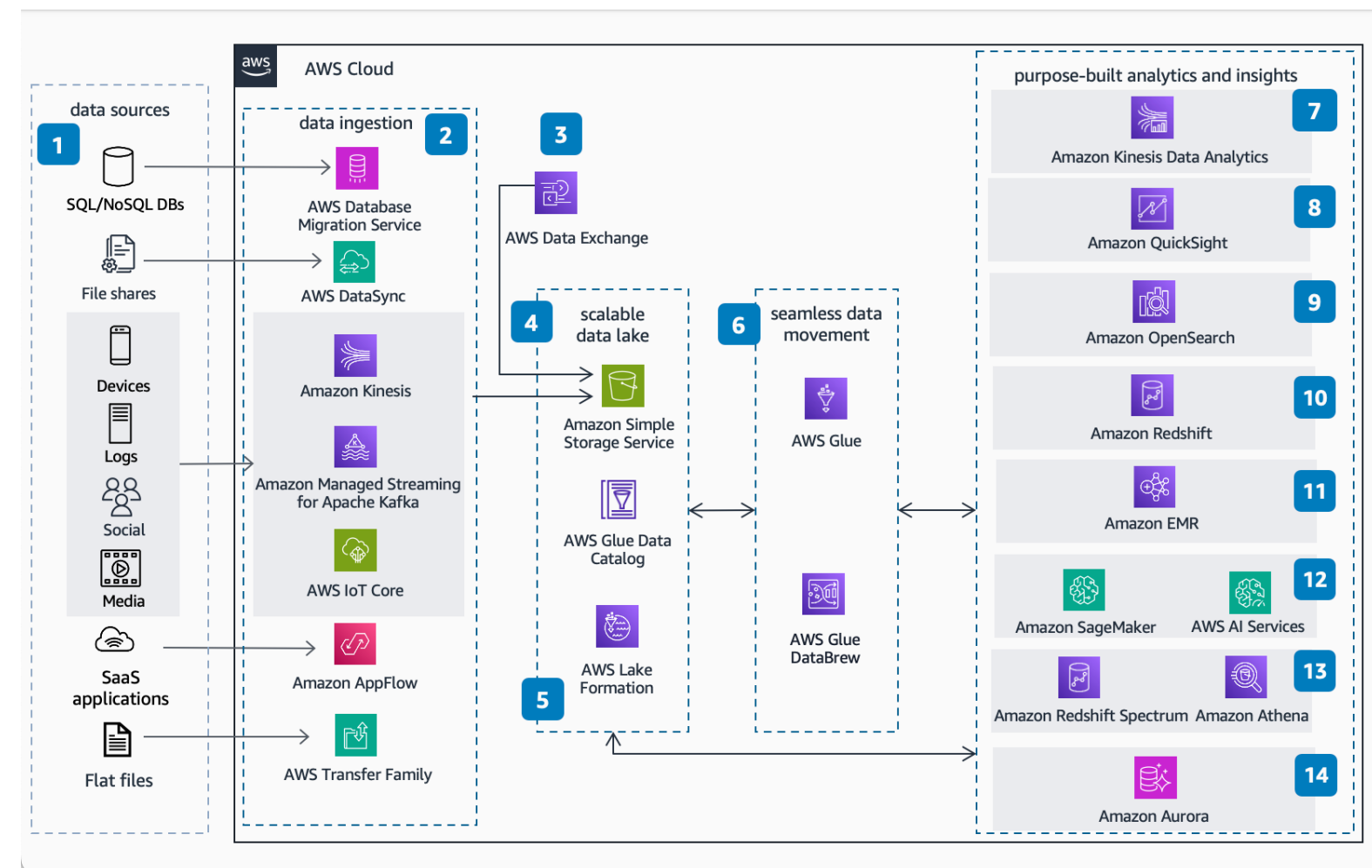
- Google Cloud Storage (GCS): Almacén de objetos escalable y duradero para almacenar grandes cantidades de datos sin estructura. Ideal para almacenar datos de respaldo, logs, imágenes, videos y otros tipos de archivos.
- Google Cloud Dataproc: Servicio administrado que permite ejecutar clústeres de Apache Spark y Hadoop en GCP. Facilita el procesamiento de datos a gran escala y la ejecución de trabajos de análisis de datos.
- BigQuery: Un almacén de datos totalmente administrado y sin servidor para análisis de datos a gran escala. Permite ejecutar consultas SQL complejas en petabytes de datos.
- Cloud Dataflow: Servicio de procesamiento de datos totalmente administrado para construir pipelines de procesamiento de datos a gran escala. Ideal para aplicaciones de procesamiento de datos en tiempo real y por lotes.
- Pub/Sub: Servicio de mensajería en la nube totalmente administrado para enviar y recibir mensajes entre aplicaciones de manera asíncrona. Perfecto para construir aplicaciones de streaming de datos y microservicios.



AWS ofrece una amplia gama de servicios para construir arquitecturas de Big Data robustas y escalables. AWS ofrece un conjunto completo de servicios de análisis que se adaptan a todas las necesidades de análisis de datos y permite a las organizaciones de todos los tamaños y sectores reinventar su negocio con datos. Desde almacenamiento y administración, gobernanza de datos, acciones y experiencias, AWS ofrece servicios especialmente diseñados que proporcionan la mejor relación precio-rendimiento, la mejor escalabilidad y el menor costo.

Componentes Clave

- **Amazon S3:** Un almacenamiento de objetos escalable y duradero para almacenar grandes volúmenes de datos sin estructura. Es ideal para almacenar datos de respaldo, logs, imágenes, videos y otros tipos de archivos.
- **Amazon EMR:** Un servicio administrado que permite ejecutar clústeres de Hadoop, Spark, Hive y otros frameworks de procesamiento de datos. Es perfecto para cargas de trabajo de procesamiento de datos a gran escala.
- **Amazon Redshift:** Un almacén de datos totalmente administrado y optimizado para consultas SQL analíticas. Ideal para análisis de datos a gran escala y creación de informes.
- **Amazon Athena:** Un servicio de análisis interactivo que permite ejecutar consultas SQL directamente sobre datos almacenados en S3. No requiere gestión de infraestructura.
- **Amazon Kinesis Data Streams:** Un servicio de streaming de datos en tiempo real que permite procesar grandes volúmenes de datos de manera continua.
- **Amazon Glue:** Un servicio de integración de datos totalmente administrado que permite descubrir, preparar y mover datos.

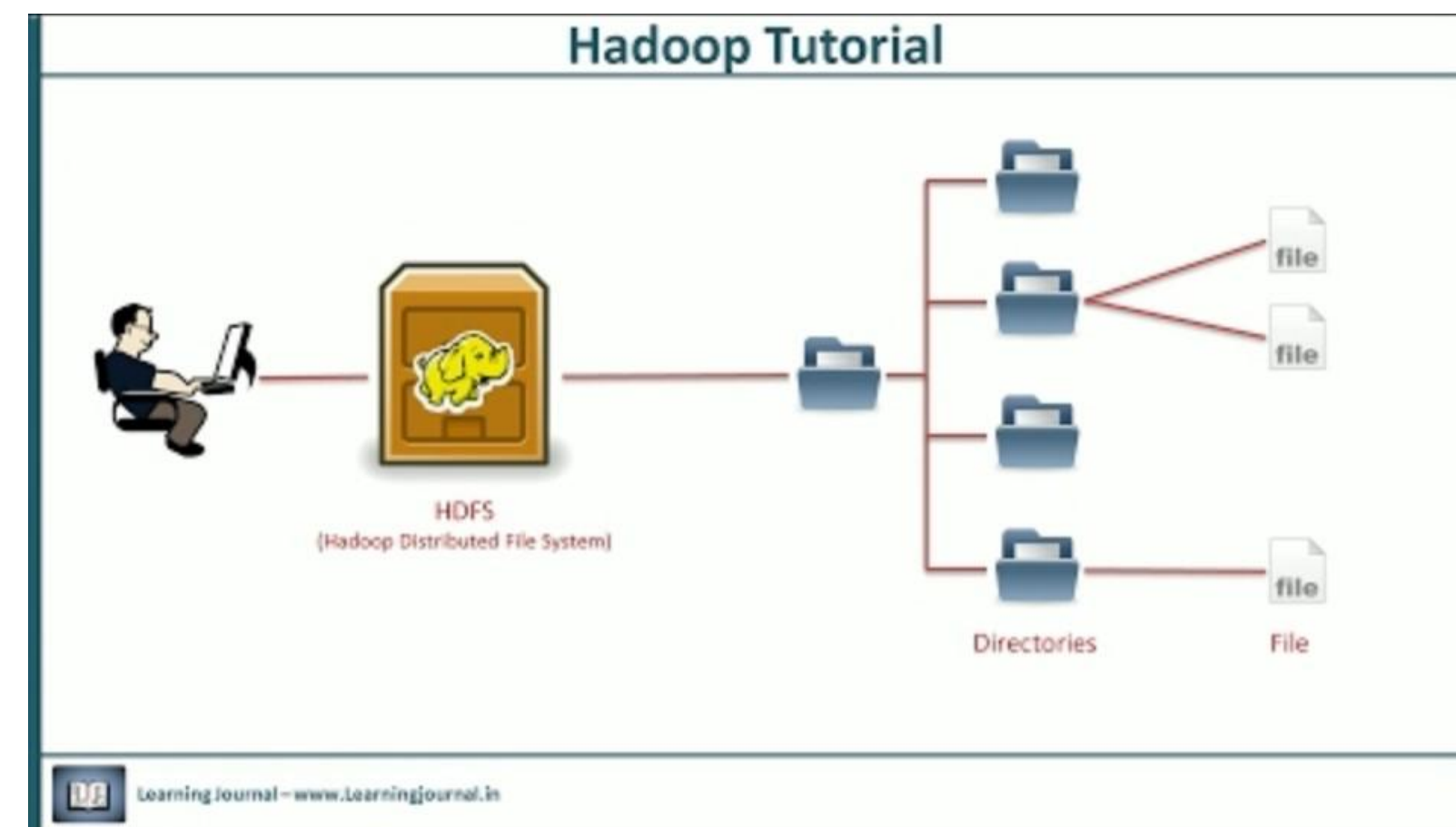


3 /SISTEMA HDFS

1.1 ¿Qué es HDFS?

HDFS (Hadoop Distributed File System) es el sistema de archivos distribuido del proyecto **Apache Hadoop**, diseñado para almacenar y gestionar grandes volúmenes de datos (Big Data) de forma eficiente y segura en un entorno distribuido. A diferencia de los sistemas de archivos tradicionales, HDFS permite el almacenamiento de datos en un clúster de máquinas, con capacidades de escalabilidad y alta disponibilidad.

- **Replicación de datos.** HDFS hace que los datos estén siempre disponibles, aunque falle uno de los ordenadores del clúster. Gracias a este sistema se pueden replicar datos y seguir su procesamiento.
- **Tolerancia a fallos.** La capacidad de replicación y de copia del sistema de archivos distribuidos hace que el sistema sea más tolerante a posibles fallos.
- **Escalabilidad.** Al funcionar a través de un clúster de ordenadores, es mucho más sencillo escalar la capacidad de procesamiento añadiendo o quitando ordenadores.
- **Rendimiento.** Al almacenar los datos en un sistema distribuido, HDFS hace que se puedan procesar de forma paralela. Esto hace que se reduzca el tiempo de procesamiento y, por tanto, se mejore el rendimiento en el mismo.

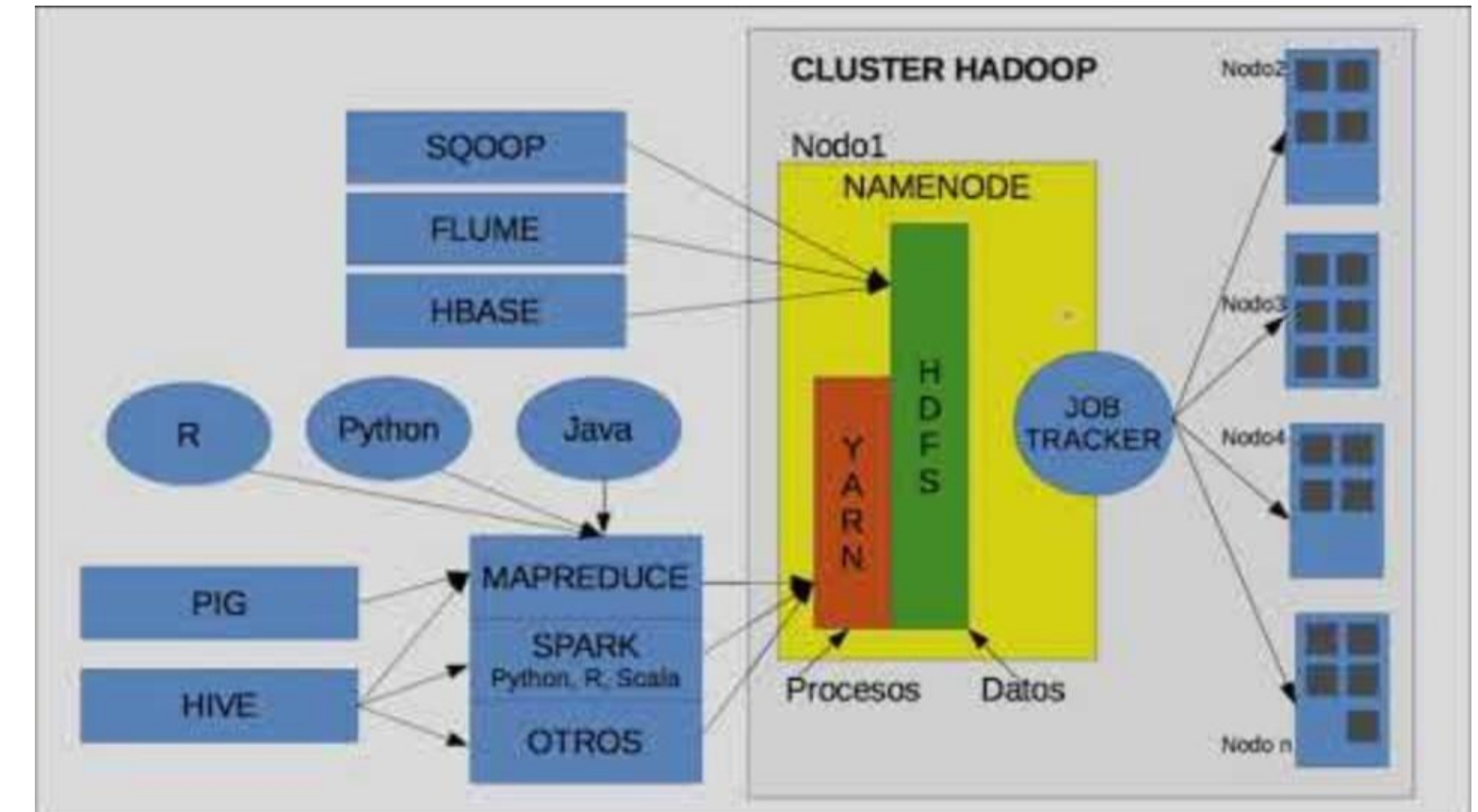


HDFS sigue un modelo maestro-esclavo, donde:

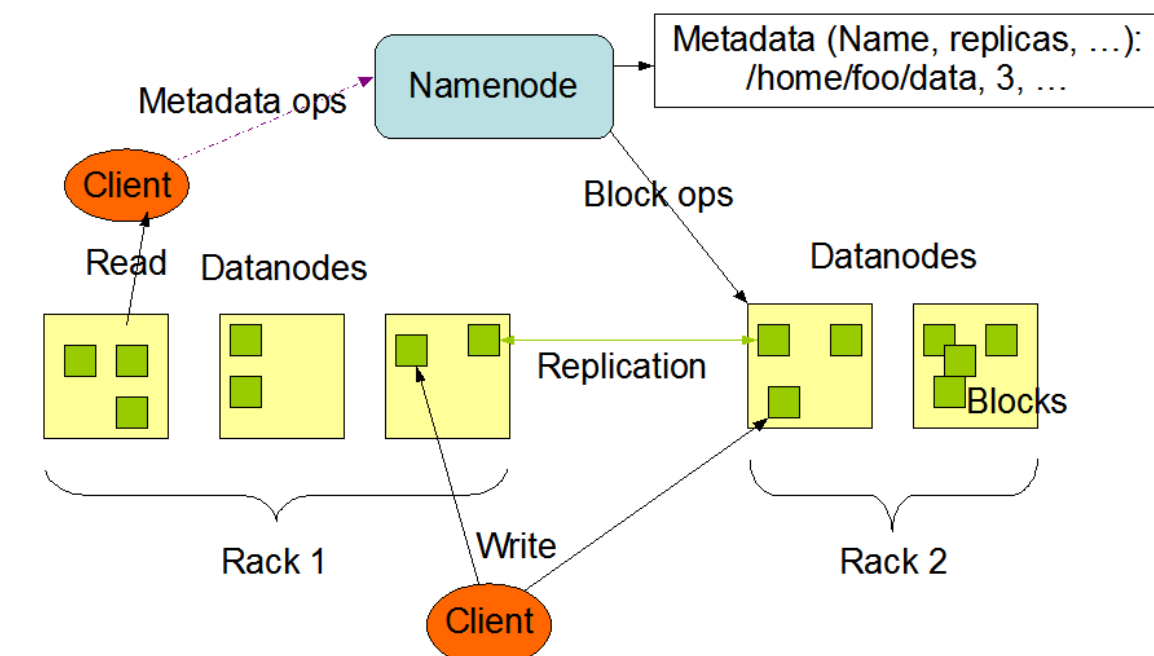
- El **NameNode** es el servidor central que administra el sistema de archivos y conoce la ubicación de los bloques de datos.
- Los **DataNodes** almacenan los datos reales en bloques y ejecutan las operaciones de lectura/escritura solicitadas por los clientes.

Características principales:

- **Distribución de datos:** Los archivos se dividen en bloques (típicamente de 128 MB) que se distribuyen a través de los DataNodes.
- **Replicación:** Cada bloque tiene múltiples copias distribuidas en varios nodos para garantizar la disponibilidad en caso de fallo.



HDFS Architecture



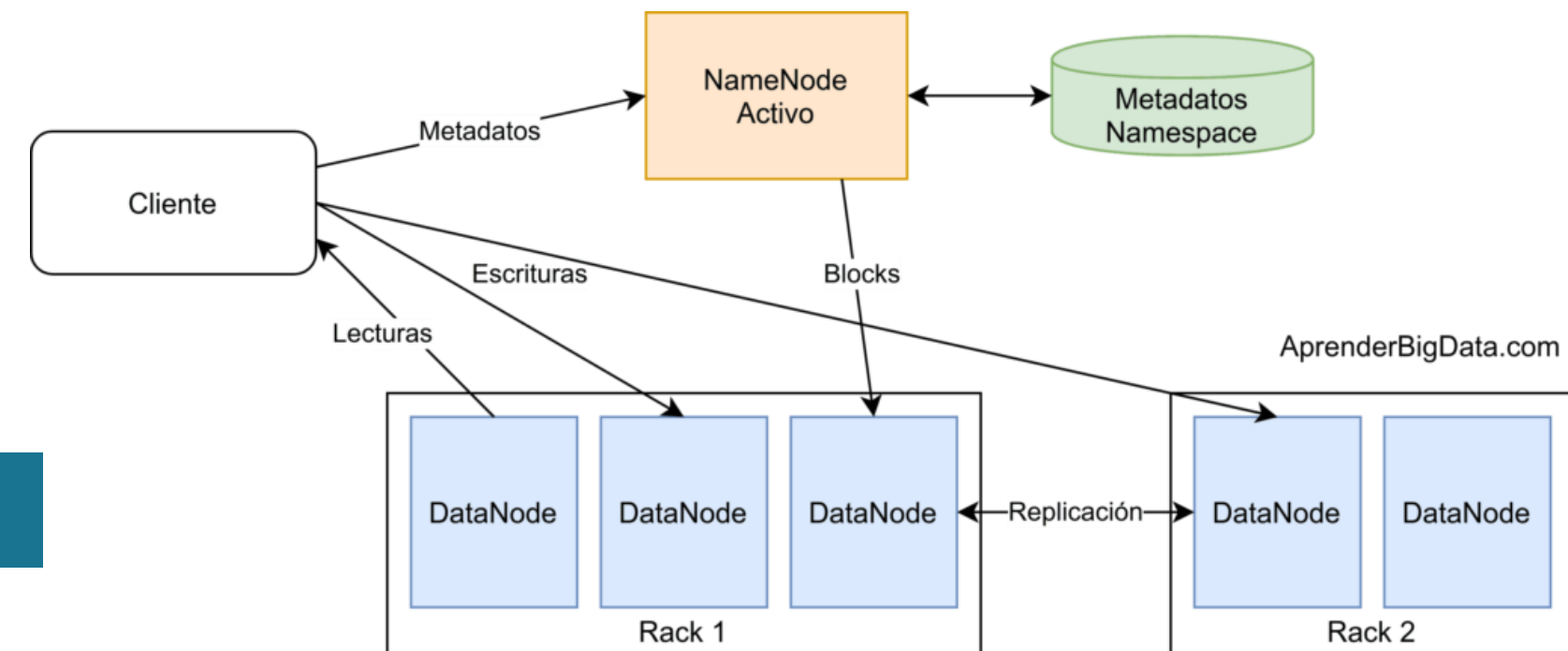
NameNode

- El NameNode es el núcleo de la arquitectura HDFS. Administra el sistema de archivos y el namespace, rastreando dónde se almacenan los bloques en el clúster. No almacena los datos reales, sino la **metadata** (información de ubicación de los bloques y estructura de archivos).
- **Funciones del NameNode:**
 - Gestionar la creación, eliminación y modificación de archivos y directorios.
 - Mantener un mapa de dónde se encuentran los bloques de datos en los DataNodes.
- **Alta disponibilidad:** Si el NameNode falla, puede ser una vulnerabilidad crítica. Por ello, se utilizan NameNodes secundarios (o standby) para tener redundancia.

DataNode

Los DataNodes son los encargados de almacenar los datos reales. Los archivos se dividen en bloques y se distribuyen entre varios DataNodes.

- **Funciones del DataNode:**
 - Almacenar y replicar bloques de datos.
 - Realizar operaciones de lectura y escritura según lo soliciten los clientes.
 - Informar periódicamente al NameNode sobre el estado de los bloques que contienen.
- **2.3 Bloques y replicación**
- HDFS divide los archivos en **bloques** de tamaño fijo (128 MB por defecto), lo que permite una distribución eficiente de los datos entre diferentes nodos del clúster.
- **Replicación:** Cada bloque se replica en varios DataNodes (típicamente tres copias por bloque) para asegurar la tolerancia a fallos. Si un nodo falla, los datos aún pueden ser accesibles desde otras réplicas.



Airbnb

Airbnb almacena y procesa grandes volúmenes de datos sobre reservas, interacciones de usuarios, precios, y ubicaciones utilizando Hadoop. Los casos de uso incluyen:

- **Análisis de datos de usuarios y reservas** para optimizar la estrategia de precios dinámicos y maximizar el ingreso para los anfitriones.
- **Modelos de machine learning** para mejorar las recomendaciones de propiedades a los usuarios, basadas en el análisis del comportamiento anterior.
- **Mejoras en la búsqueda de propiedades:** Utiliza Hadoop para analizar las consultas de búsqueda y mejorar los algoritmos que sugieren los lugares más relevantes.



Netflix

Netflix usa Hadoop para gestionar y analizar datos masivos provenientes de las interacciones de los usuarios con su plataforma. Los principales casos de uso incluyen:

- **Recomendaciones personalizadas:** Netflix utiliza Hadoop para analizar el historial de visualizaciones y patrones de comportamiento de los usuarios, mejorando sus algoritmos de recomendación de contenido.
- **Análisis predictivo:** Hadoop permite a Netflix predecir qué contenido será popular y ajustar su catálogo de películas y series en consecuencia.
- **Optimización de la infraestructura de red:** Analizando datos de streaming en tiempo real para mejorar la calidad de video y reducir tiempos de buffering.



2/ Apache Kafka

Apache Kafka es una **plataforma de mensajería distribuida** diseñada para manejar el procesamiento de flujos de datos en tiempo real. Kafka permite publicar, suscribirse, almacenar y procesar transmisiones de datos de manera eficiente y escalable. A diferencia de los sistemas de mensajería tradicionales, que son generalmente más simples y menos orientados a manejar grandes volúmenes de datos, Kafka se destaca por su alta **durabilidad, escalabilidad horizontal y bajo acoplamiento** entre los productores y consumidores de datos.

En términos simples, Kafka actúa como un "intermediario" o "buffer" que facilita la transmisión de datos (mensajes) entre sistemas que producen datos (producers) y sistemas que consumen o procesan esos datos (consumers).

Apache Kafka fue creado en **2011** por ingenieros de LinkedIn (Jay Kreps, Neha Narkhede y Jun Rao) para resolver un problema particular: la creciente necesidad de procesar enormes cantidades de datos generados en tiempo real dentro de la red social profesional. LinkedIn se enfrentaba a desafíos en la administración y procesamiento de flujos de eventos y datos de diversas fuentes, como actividad de usuarios, métricas operativas y registros de aplicaciones.

Antes de Kafka, LinkedIn, al igual que muchas otras empresas, usaba una combinación de bases de datos y herramientas tradicionales para manejar sus datos, pero estas soluciones no eran suficientemente rápidas ni eficientes para manejar el volumen creciente de información. Se necesitaba una solución que pudiera:

- **Escalar** de forma horizontal en cientos de servidores.
- Ofrecer **alta tolerancia a fallos**.
- Manejar grandes volúmenes de datos en tiempo real sin comprometer el rendimiento.
- Proporcionar una **baja latencia** en la entrega de mensajes.

Para resolver estos problemas, Apache Kafka fue diseñado con los principios de:

1.Almacenamiento distribuido: Los datos se distribuyen en múltiples servidores o "brokers", y se pueden replicar para aumentar la durabilidad.

2.Procesamiento en tiempo real: Kafka está optimizado para manejar y procesar datos a medida que se generan, en lugar de ser un sistema de procesamiento por lotes.

3.Altísima escalabilidad: Kafka puede manejar millones de mensajes por segundo con una infraestructura adecuada, lo que lo convierte en una herramienta adecuada para sistemas de gran escala.

4.Persistencia: A diferencia de muchos sistemas de mensajería, Kafka almacena mensajes durante períodos de tiempo configurables, lo que permite a los consumidores leer mensajes antiguos según sea necesario.



Componentes Principales de Apache Kafka

Los principales actores en el sistema Kafka son:

a. Producers (Productores)

Los **producers** son las aplicaciones o servicios que **generan y envían mensajes** a Kafka. Los productores publican los mensajes en **topics** específicos. Algunas características clave de los producers:

Elección de particiones: Un productor puede decidir en qué partición de un topic específico almacenar un mensaje. Puede hacerlo aleatoriamente o utilizando una clave para garantizar que los mensajes relacionados vayan a la misma partición.

Confirmación de envío (acks): Los productores pueden configurar cómo Kafka debe confirmar la recepción de los mensajes. Por ejemplo, pueden esperar confirmación de todos los brokers o solo de uno para equilibrar rendimiento y durabilidad.

b. Consumers (Consumidores)

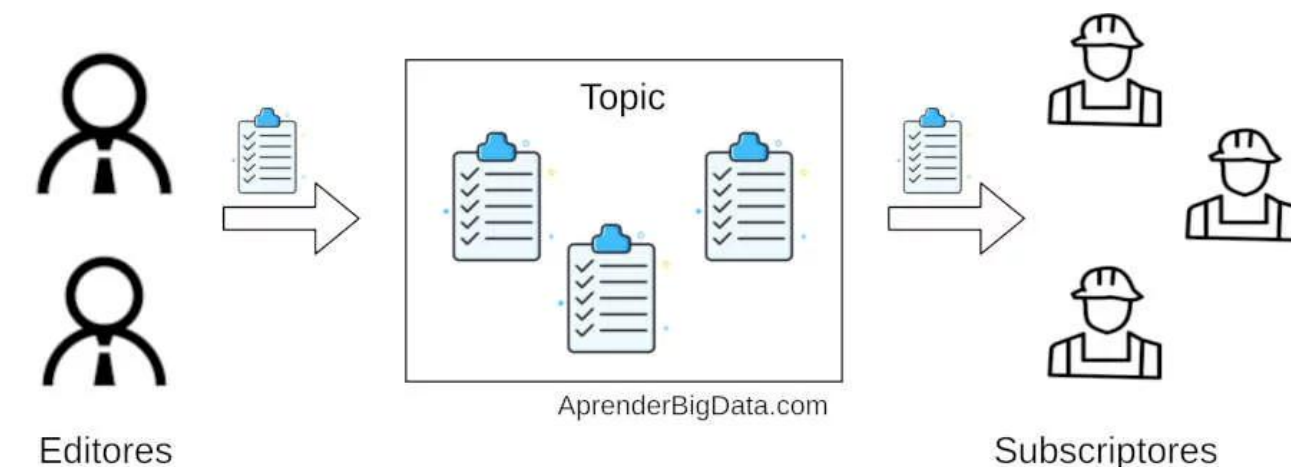
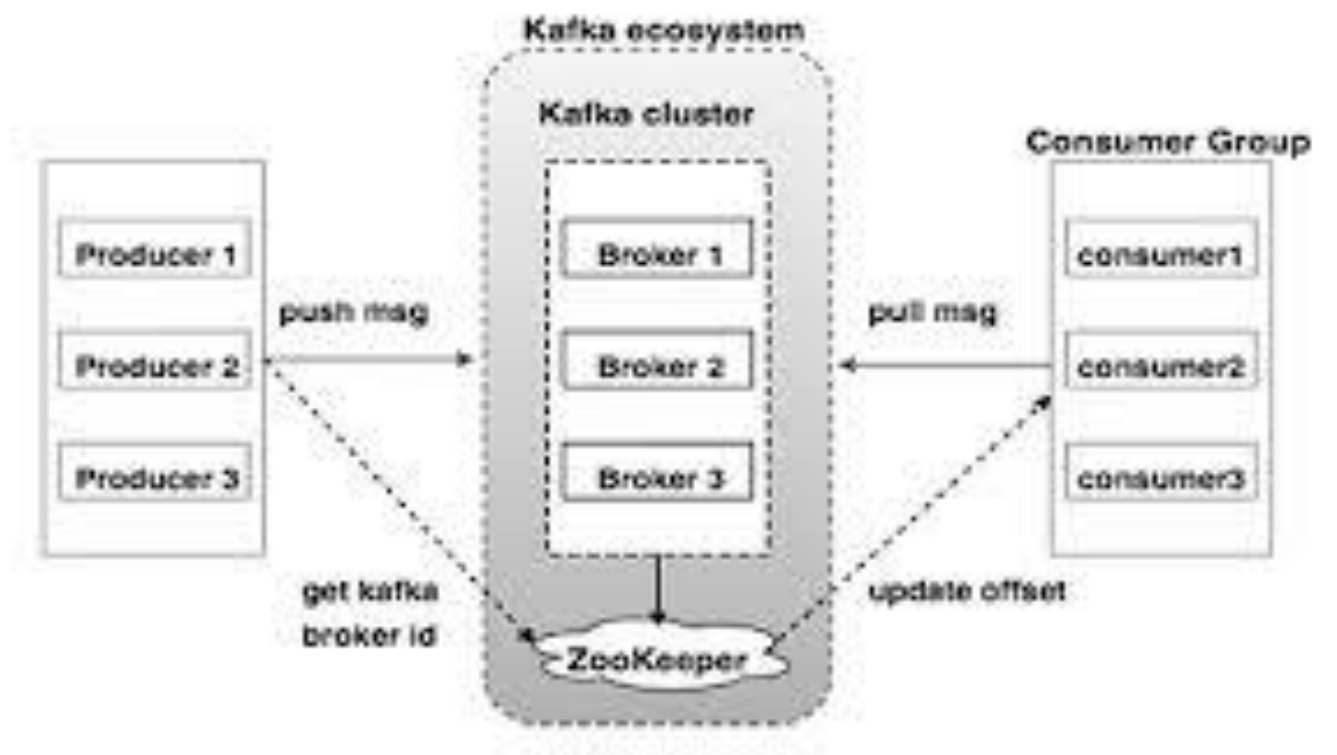
Los **consumers** son aplicaciones que **leen y procesan los mensajes** de Kafka. Se suscriben a uno o varios **topics** y consumen los mensajes en el orden en que fueron publicados. Las características principales son:

- **Consumer Groups:** Los consumidores pueden agruparse en **grupos de consumidores**. Cada grupo de consumidores puede leer mensajes de manera independiente. Kafka distribuye automáticamente las particiones de un topic entre los consumidores dentro de un mismo grupo, lo que permite un procesamiento paralelo.
- **Offsets:** Los consumidores mantienen un registro llamado **offset** que marca el último mensaje leído en una partición. Esto permite que los consumidores reanuden el procesamiento desde donde lo dejaron en caso de un fallo.

c. Brokers (Nodos del Cluster)

Los **brokers** son los servidores que forman el **cluster de Kafka**. Cada broker se encarga de:

- **Almacenar mensajes:** Los brokers almacenan los mensajes de las particiones de los topics. Cada mensaje tiene un **offset** único dentro de la partición que permite a los consumidores leer los mensajes en orden.
- **Gestión de particiones:** Un broker puede ser el líder de una partición, lo que significa que coordina las operaciones de lectura y escritura de esa partición. Los otros brokers mantendrán réplicas de los datos para asegurar la alta disponibilidad.



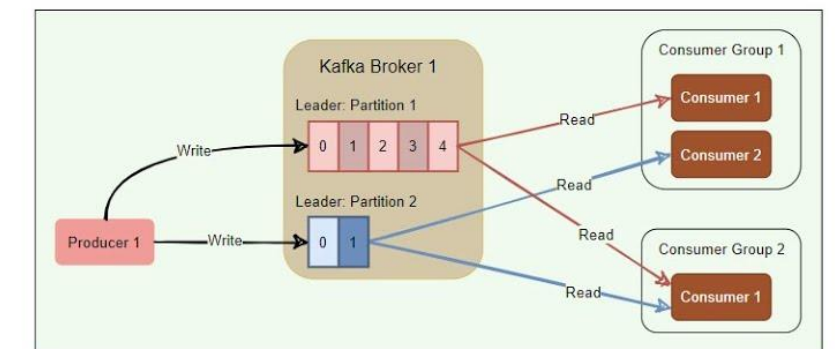
d. Zookeeper (o KRaft en nuevas versiones)

Zookeeper es un sistema de coordinación distribuida que Kafka utilizaba tradicionalmente para tareas de administración del cluster, como:

- **Almacenamiento de metadatos** sobre brokers, topics, particiones y consumidores.
- **Elección del broker líder** para cada partición.
- **Monitoreo de la salud del cluster.**

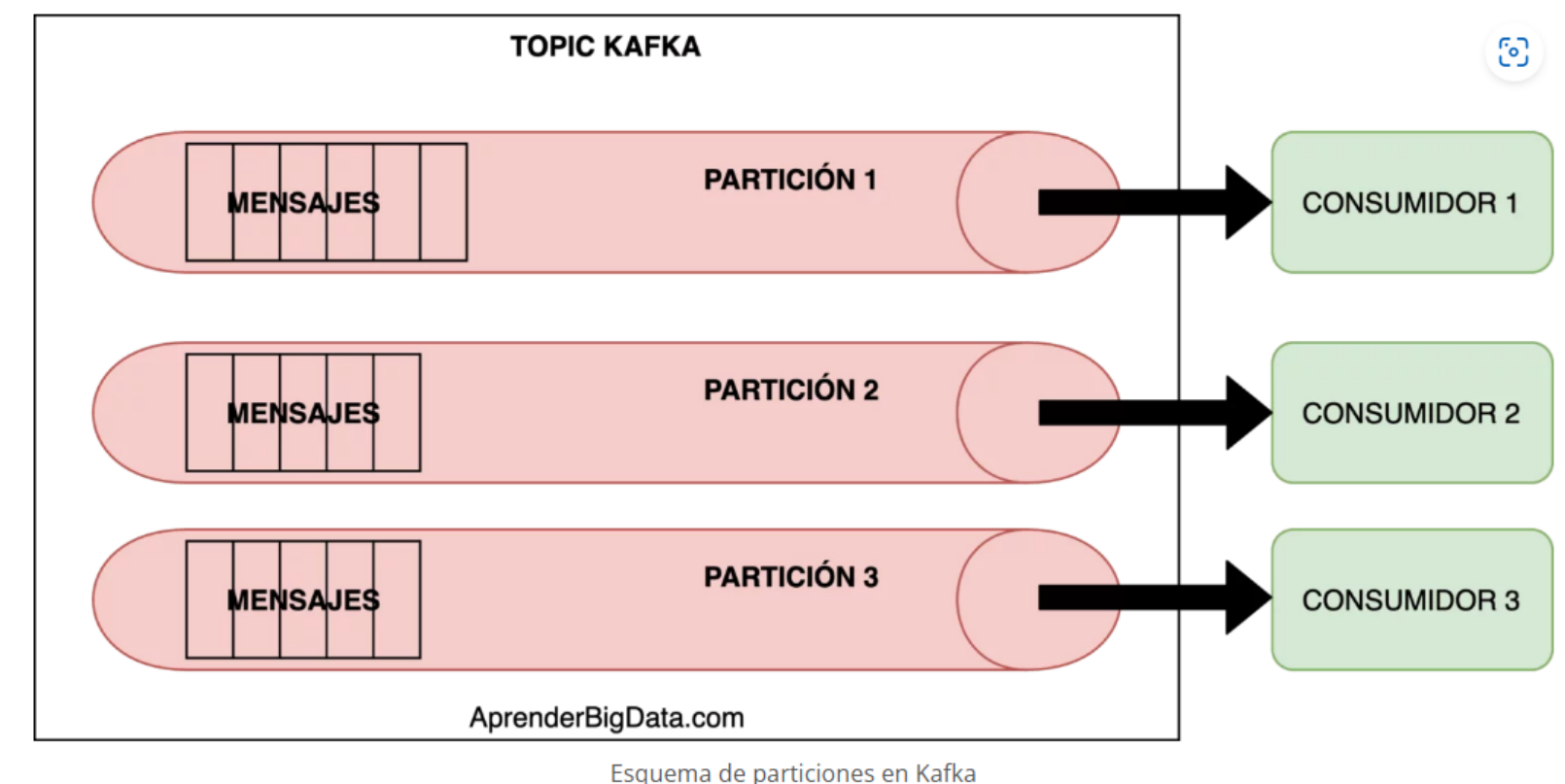
Sin embargo, en las versiones más recientes de Kafka, se ha empezado a reemplazar Zookeeper con una nueva arquitectura llamada **KRaft**, que es más eficiente y elimina la dependencia de Zookeeper para la coordinación del cluster.

Kafka:- Role of Zookeeper



Un **topic** en Kafka es una categoría o feed específico al que los productores envían mensajes y los consumidores se suscriben. Un topic se subdivide en múltiples **particiones** para mejorar la escalabilidad y permitir la paralelización del procesamiento. Cada partición actúa como un log de datos ordenado:

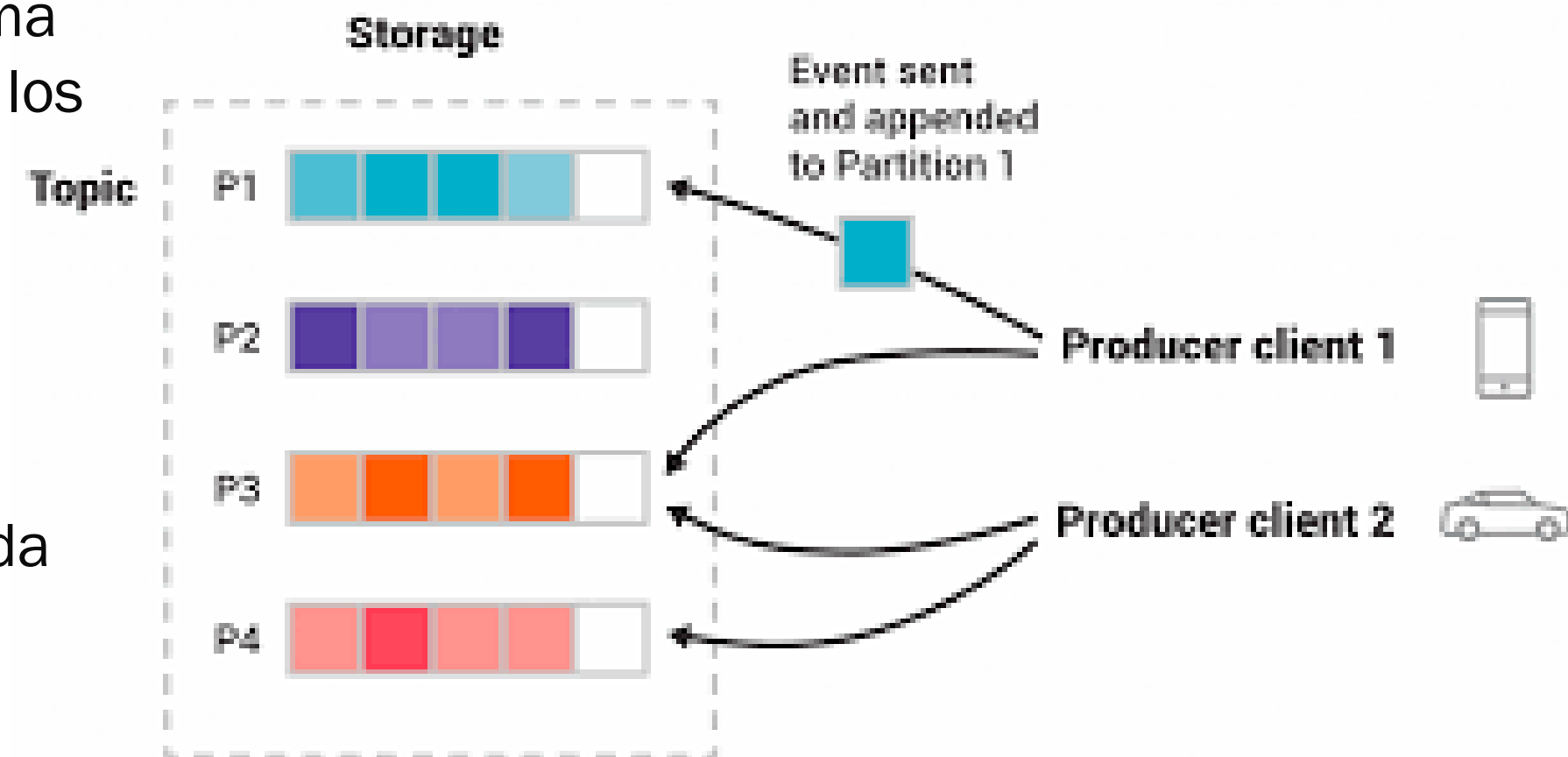
- **Particionamiento:** Cuando un productor envía un mensaje a un topic, Kafka distribuye los mensajes en varias particiones. Este esquema de particionamiento permite que diferentes consumidores lean las particiones en paralelo, mejorando el rendimiento.
- **Particiones como logs ordenados:** Los mensajes dentro de una partición son **ordenados secuencialmente** y cada mensaje tiene un **offset**, que es un identificador único de su posición dentro de la partición. Los consumidores pueden leer los mensajes en el orden en que se almacenaron, y cada consumidor mantiene un registro de hasta dónde ha leído en cada partición utilizando el offset.
- **Réplicas:** Para garantizar la **tolerancia a fallos**, Kafka permite crear múltiples réplicas de una partición en otros brokers. Un broker actúa como el **líder** de una partición, mientras que otros brokers mantienen réplicas de respaldo. Si el broker líder falla, uno de los brokers con réplicas toma el control.



4. Particiones y Réplicas

Cada **topic** en Kafka se divide en particiones, y cada partición se almacena en un broker del cluster. Kafka implementa un sistema de **réplicas** para garantizar la durabilidad y la disponibilidad de los datos. Las particiones y réplicas se organizan de la siguiente manera:

- **Partición líder y réplicas:** Dentro del cluster, cada partición tiene un **líder** y varias **réplicas**. El líder es responsable de manejar todas las operaciones de lectura y escritura de la partición. Las réplicas solo mantienen una copia sincronizada de los datos.
- **Failover automático:** Si el broker líder de una partición falla, uno de los brokers con réplicas asume el rol de líder de forma automática, asegurando la continuidad del servicio sin pérdida de datos.



INTRODUCCIÓN APACHE KAFKA

 Aprender BIG DATA
AprenderBigData.com

Apache Kafka para principiantes

por Andrés Gómez Ferrer

 **KEEPCODING**
Tech School



3/ Confluent

Se trata de una plataforma de streaming basada en Apache Kafka cuyo objetivo es **ayudar a los diferentes roles en la organización en los proyectos de intercambio de información en tiempo real**.

Para ello, desde un punto de vista de arquitectura tecnológica, mejora las capacidades nativas base de Apache Kafka en las siguientes perspectivas:

- Capacidades de Streaming Enterprise
- Conectividad y Gobierno del Dato
- Procesamiento Avanzado
- Monitorización y Gestión
- Seguridad
- Arquitecturas MultiSite y Apoyo en la evolución hacia el Cloud



CONFLUENT

Es capaz de:

- Integrar fuentes productoras y consumidoras de eventos de diferente naturaleza
- Gestionar las capacidades Real Time para transporte de altos volúmenes de datos (Megabytes por segundo)
- Garantizar la resiliencia, escalabilidad y secuencialidad en la transferencia de eventos.
- Colaborar con el resto de los componentes de arquitectura para la gestión de los metadatos, la distribución de carga dentro del clúster.
- Mediante Confluent Server se consiguen maximizar las capacidades de Streaming



Fuente: Confluent

Explore nuestros casos de uso más populares

Detección del fraude

Microservicio s event-driven

Pipelines de bases de datos

Integración del mainframe

Pipelines de almacenes de datos

Integración de mensajería

Optimización de SIEM

Análisis de IoT en tiempo real

Reabastecimiento en tiempo real

Redes autónomas

IA generativa

Gestión de riesgos

Análisis en tiempo real

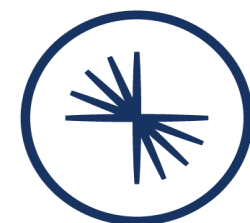
Integración de Security Lake

Análisis predictivo

Inventario en tiempo real

Análisis de vendedores en tiempo real

Pagos en tiempo real



CONFLUENT

3/ BBDD NoSQL

El término **NoSQL** significa "Not Only SQL" o "No SQL", refiriéndose a un conjunto de sistemas de gestión de bases de datos que no siguen el modelo tradicional de bases de datos relacionales basadas en SQL. Surgieron como respuesta a las limitaciones de escalabilidad y flexibilidad de las bases de datos relacionales, especialmente cuando las empresas empezaron a enfrentarse a volúmenes masivos de datos, como en el caso de redes sociales, comercio electrónico, y análisis en tiempo real.

Características principales de NoSQL:

- **Esquemas flexibles:** A diferencia de las bases de datos SQL, las bases de datos NoSQL no requieren un esquema fijo. Los datos pueden tener diferentes estructuras dentro de la misma colección o tabla.
- **Distribución horizontal:** Permiten la partición de datos (sharding) en múltiples servidores para mejorar el rendimiento y la escalabilidad.
- **Alta disponibilidad y tolerancia a fallos:** Están diseñadas para entornos distribuidos, lo que las hace más resistentes a fallos de hardware o red.



Las bases de datos NoSQL surgieron como una solución para manejar las necesidades de las aplicaciones modernas, que se enfrentan a un gran volumen, variedad y velocidad de datos (las 3 V's del Big Data). Algunas de las necesidades clave que resuelven son:

- **Escalabilidad horizontal:** Las bases de datos NoSQL permiten distribuir los datos en múltiples nodos de forma eficiente, algo que las bases de datos relacionales no hacen tan bien. Esto es fundamental en aplicaciones como redes sociales, donde la cantidad de usuarios y datos crece exponencialmente.
- **Flexibilidad de almacenamiento:** Muchas aplicaciones actuales manejan datos no estructurados o semi-estructurados, como JSON, XML o archivos multimedia. Las bases de datos NoSQL permiten almacenar estos datos sin la necesidad de un esquema fijo, lo que reduce la complejidad.
- **Rendimiento en tiempo real:** Las aplicaciones como los sistemas de recomendación, el análisis en tiempo real o la publicidad digital requieren tiempos de respuesta muy rápidos con grandes volúmenes de datos. Las bases de datos NoSQL están optimizadas para estas operaciones rápidas de lectura y escritura.

Ventajas de las bases de datos NoSQL

Flexibilidad

Las bases de datos NoSQL pueden almacenar y administrar datos en varios formatos, incluidos JSON, XML, pares clave-valor y más. Esta flexibilidad permite a las organizaciones adaptarse a los requisitos de datos cambiantes sin alterar el esquema de su base de datos.

Escalabilidad

Las bases de datos NoSQL están hechas para crecer de forma horizontal. Pueden trabajar con mucha información y mucha gente repartiendo los datos entre varios puntos o grupos, lo que asegura que la velocidad no se pierda a medida que hay más demanda.

Alta disponibilidad

Tienen sistemas incorporados para asegurar que no se caigan, ni siquiera cuando hay problemas con los equipos o la conexión. Esto es muy importante para aplicaciones que no pueden parar, que necesitan estar funcionando siempre.

Funcionalidad mejorada

Las bases de datos NoSQL tienen tipos de datos que se adaptan a cada situación, que nos ofrecen modelos de datos distintos y formas de trabajar con ellos muy sencillas.

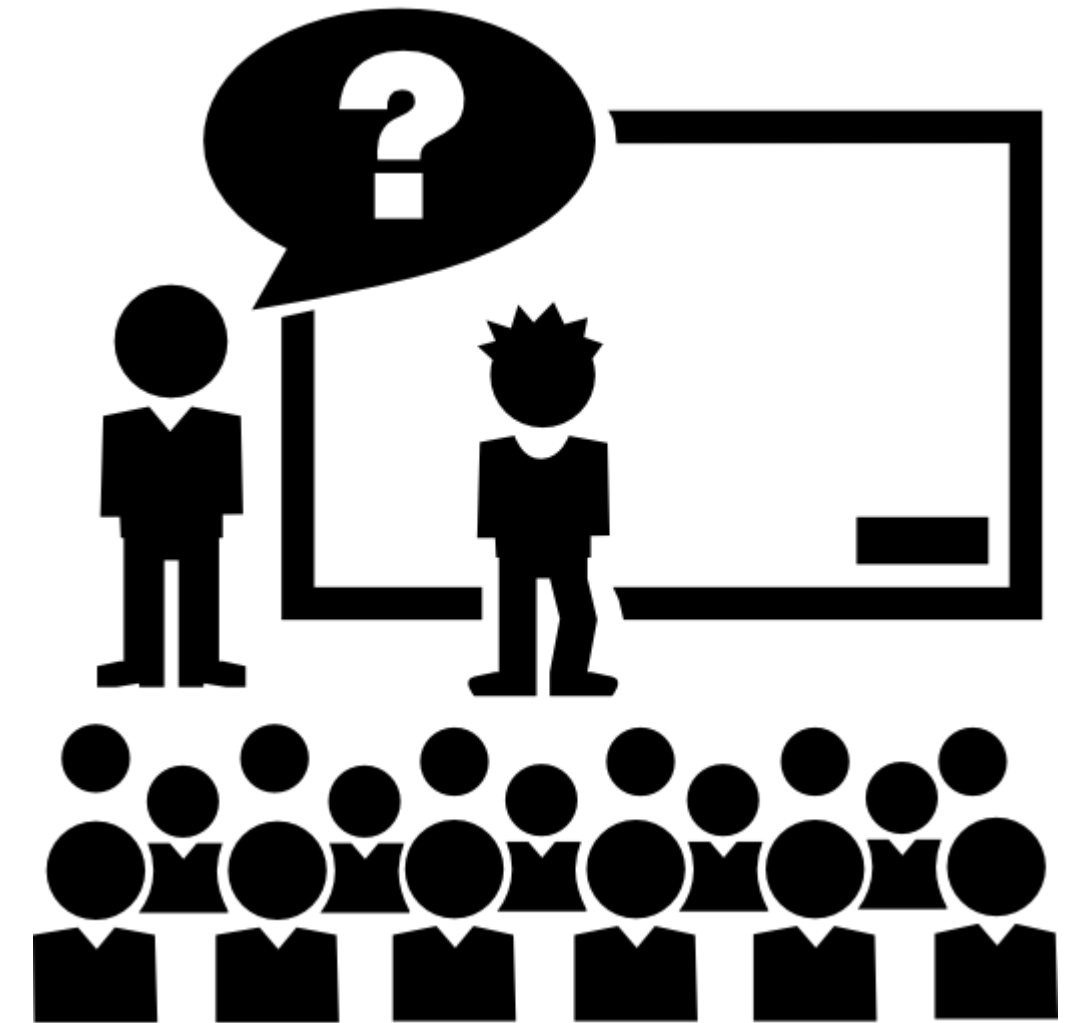
SQL (Structured Query Language) ha sido la base de los sistemas de gestión de bases de datos relacionales (RDBMS) durante décadas, proporcionando un modelo estructurado y estandarizado para manejar datos.

- **Estructura de datos:**
 - **SQL:** Basadas en tablas con filas y columnas, lo que requiere que todos los datos sigan un esquema predefinido.
 - **NoSQL:** Pueden almacenar datos no estructurados o semi-estructurados (JSON, documentos, pares clave-valor), lo que las hace más flexibles.
- **Escalabilidad:**
 - **SQL:** Escala verticalmente (añadiendo más recursos a un solo servidor).
 - **NoSQL:** Escala horizontalmente (añadiendo más servidores), lo que es ideal para aplicaciones con un gran volumen de datos.
- **Consistencia vs Disponibilidad:** Introducir el **Teorema CAP** (Consistencia, Disponibilidad y Tolerancia a particiones).
 - **SQL:** En su mayoría, priorizan la **consistencia fuerte** (todos los usuarios ven la misma versión de los datos en cualquier momento).
 - **NoSQL:** A menudo optan por **disponibilidad y tolerancia a particiones** en sistemas distribuidos, aceptando cierto grado de inconsistencia temporal (consistencia eventual).

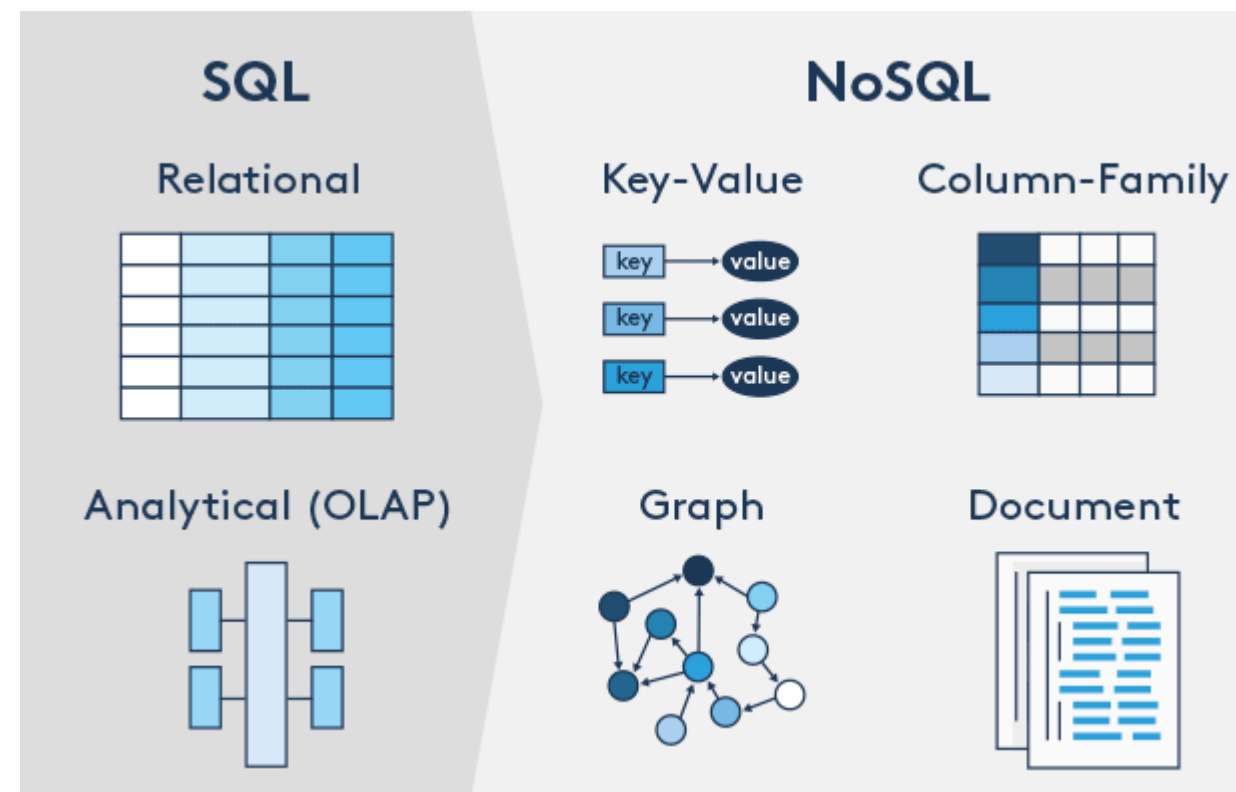
Característica	SQL	NoSQL
Estructura de datos	Esquema fijo (tablas)	Esquema flexible
Consistencia	Consistencia fuerte	Consistencia eventual
Escalabilidad	Vertical	Horizontal
Tipo de transacciones	ACID (transacciones completas)	BASE (transacciones más ligeras)
Aplicaciones típicas	ERP, CRM, contabilidad	Redes sociales, Big Data

Pregunta a la clase: "¿Cuáles de las aplicaciones que utilizan diariamente creen que usan NoSQL?"

(Ejemplos: redes sociales como Facebook o Twitter, aplicaciones de mensajería como WhatsApp, plataformas de streaming como Netflix, AEAT)".



En el ecosistema de bases de datos NoSQL, existen cuatro tipos principales, cada uno de ellos diseñado para casos de uso específicos según la estructura y las necesidades del tipo de datos que manejan. Estos son: bases de datos de **clave-valor**, **documentos**, **columnar** y **grafos**.



BBDD Clave-Valor

Una base de datos clave-valor es el modelo más simple de base de datos NoSQL. En este tipo de bases de datos, los datos se almacenan como pares **clave-valor**, donde una clave única identifica a un valor asociado. Este modelo es muy eficiente para operaciones de lectura y escritura rápidas, lo que lo hace ideal para caché y sistemas de alto rendimiento.

Funcionamiento: El acceso a los datos se realiza a través de una clave única, como si fuera un gran diccionario o mapa en programación.

Casos de uso:

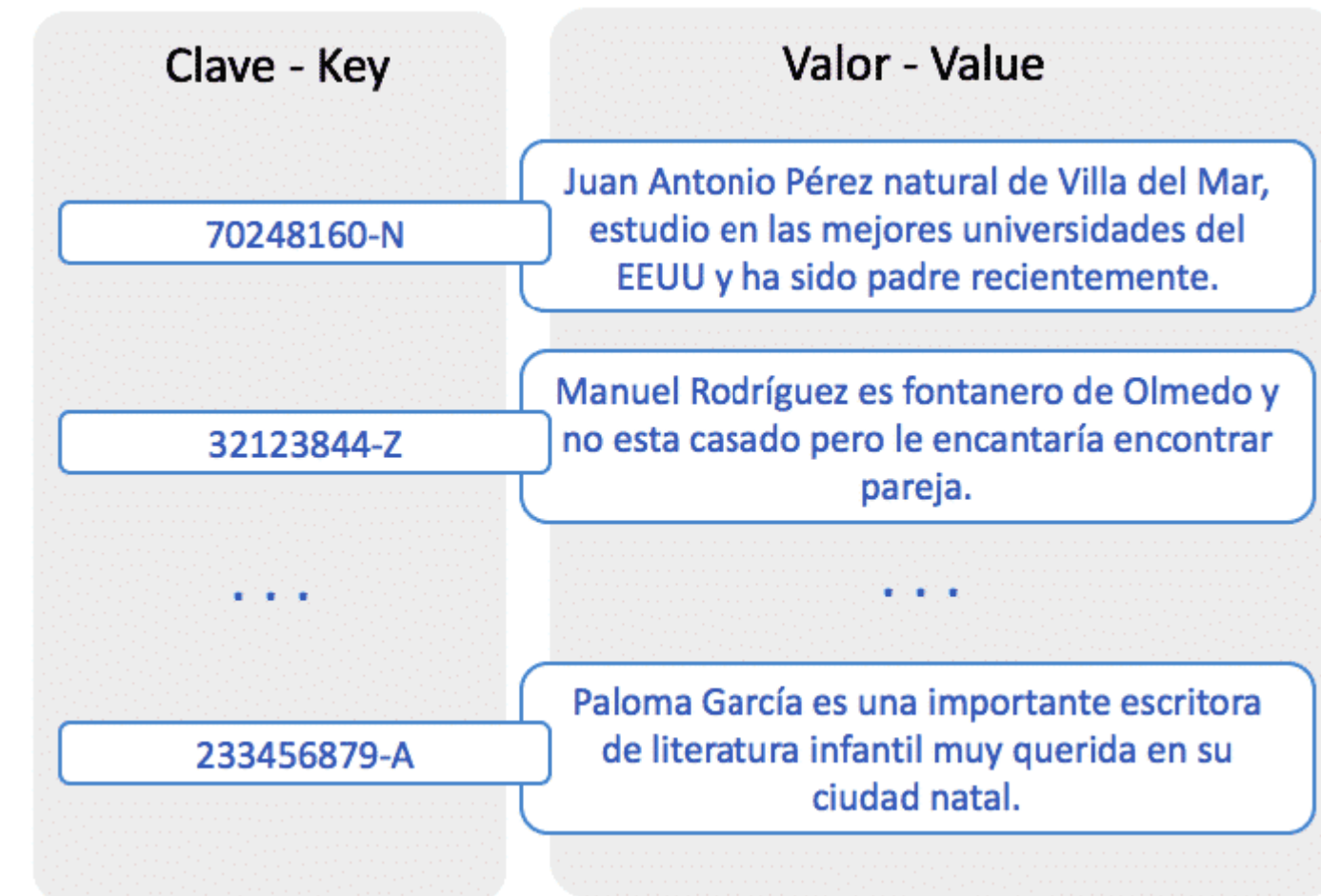
Sistemas de caché: Donde se almacenan resultados de consultas costosas para acceso rápido (ej. Redis).

Sesiones de usuario: Donde se guarda el estado del usuario en aplicaciones web.

Ejemplos:

Redis: Un sistema de base de datos en memoria que se usa principalmente para almacenar estructuras de datos en caché.

Amazon DynamoDB: Una base de datos NoSQL administrada que permite una escalabilidad masiva con baja latencia.



json

Copiar código

```
{
  "user123": "John Doe",
  "session567": "active",
  "cart789": "item1, item2, item3"
}
```

Las bases de datos de documentos almacenan los datos en formato de documentos, habitualmente en **JSON**, **BSON** o **XML**. Cada documento es una entidad autónoma que puede tener un esquema flexible, lo que significa que los diferentes documentos en una misma colección no necesitan tener el mismo formato.

- **Funcionamiento:** Los datos se organizan en documentos, cada uno de los cuales puede tener diferentes campos. Los documentos se agrupan en colecciones y se pueden indexar para facilitar consultas rápidas.

Casos de uso:

- **Aplicaciones web:** Que manejan grandes cantidades de datos semiestructurados, como perfiles de usuario, registros de actividad, o información de productos.
- **Sistemas de gestión de contenido (CMS):** Donde los datos tienen estructura flexible o cambiante.

Ejemplos:

- **MongoDB:** Es el sistema de base de datos de documentos más popular, ideal para aplicaciones que requieren alta flexibilidad en el manejo de datos.
- **CouchDB:** Utiliza JSON para almacenar datos y ofrece una API HTTP para realizar consultas.

Visualización (ejemplo de MongoDB):

```
json Copiar código
{
  "_id": "603c9c2b2f1f4f07a44f05a9",
  "name": "John Doe",
  "email": "johndoe@example.com",
  "orders": [
    { "order_id": "001", "total": 100, "status": "shipped" },
    { "order_id": "002", "total": 200, "status": "pending" }
  ]
}
```

Las bases de datos columnar, también conocidas como **almacenes de columnas**, almacenan los datos en columnas en lugar de filas. Este tipo de bases de datos es eficiente para realizar consultas en grandes volúmenes de datos y se usa comúnmente en aplicaciones analíticas y de Big Data, donde las consultas requieren leer muchas columnas a la vez.

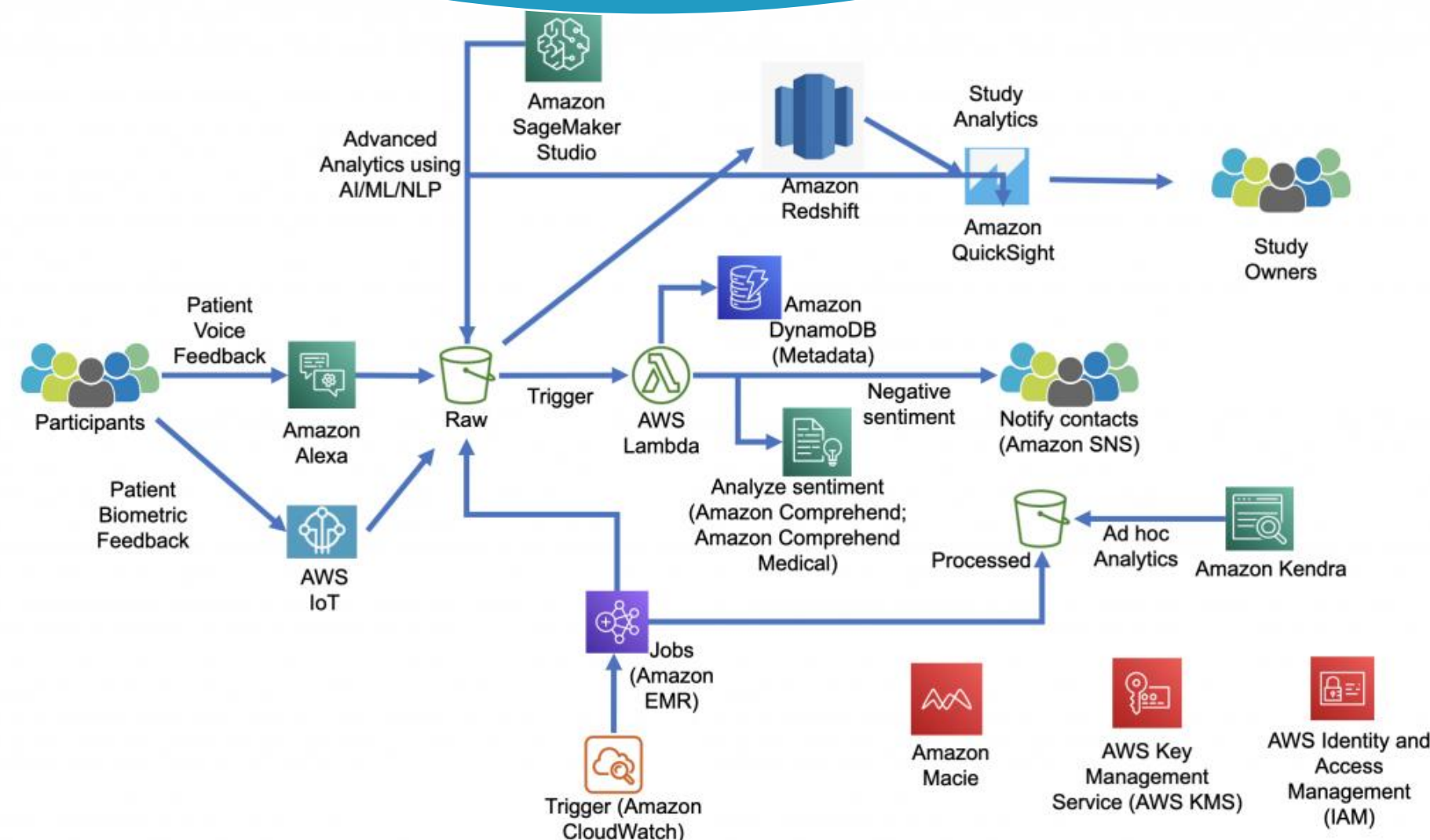
Funcionamiento: Los datos se agrupan por columnas en lugar de por filas, lo que permite leer solo las columnas necesarias para una consulta, mejorando el rendimiento en operaciones de lectura intensiva.

Casos de uso:

- **Analítica y Big Data:** En sistemas que manejan grandes volúmenes de datos y requieren realizar análisis rápido de ciertos campos (ej. **Hadoop HBase**).
- **Sistemas de recomendación:** Donde es necesario procesar grandes volúmenes de datos y buscar correlaciones rápidamente.

Ejemplos:

- **Apache Cassandra:** Optimizada para manejar grandes cantidades de datos a través de muchos servidores.
- **HBase:** Una base de datos distribuida construida sobre Hadoop
- **Amazon Redshift** es un servicio de **almacenamiento de datos en la nube** diseñado y administrado por Amazon Web Services (AWS).



Las bases de datos de grafos están diseñadas para almacenar relaciones entre entidades. Los datos se organizan en nodos (entidades) y aristas (relaciones), y este enfoque es ideal para representar y consultar redes complejas, como redes sociales o mapas de tráfico.

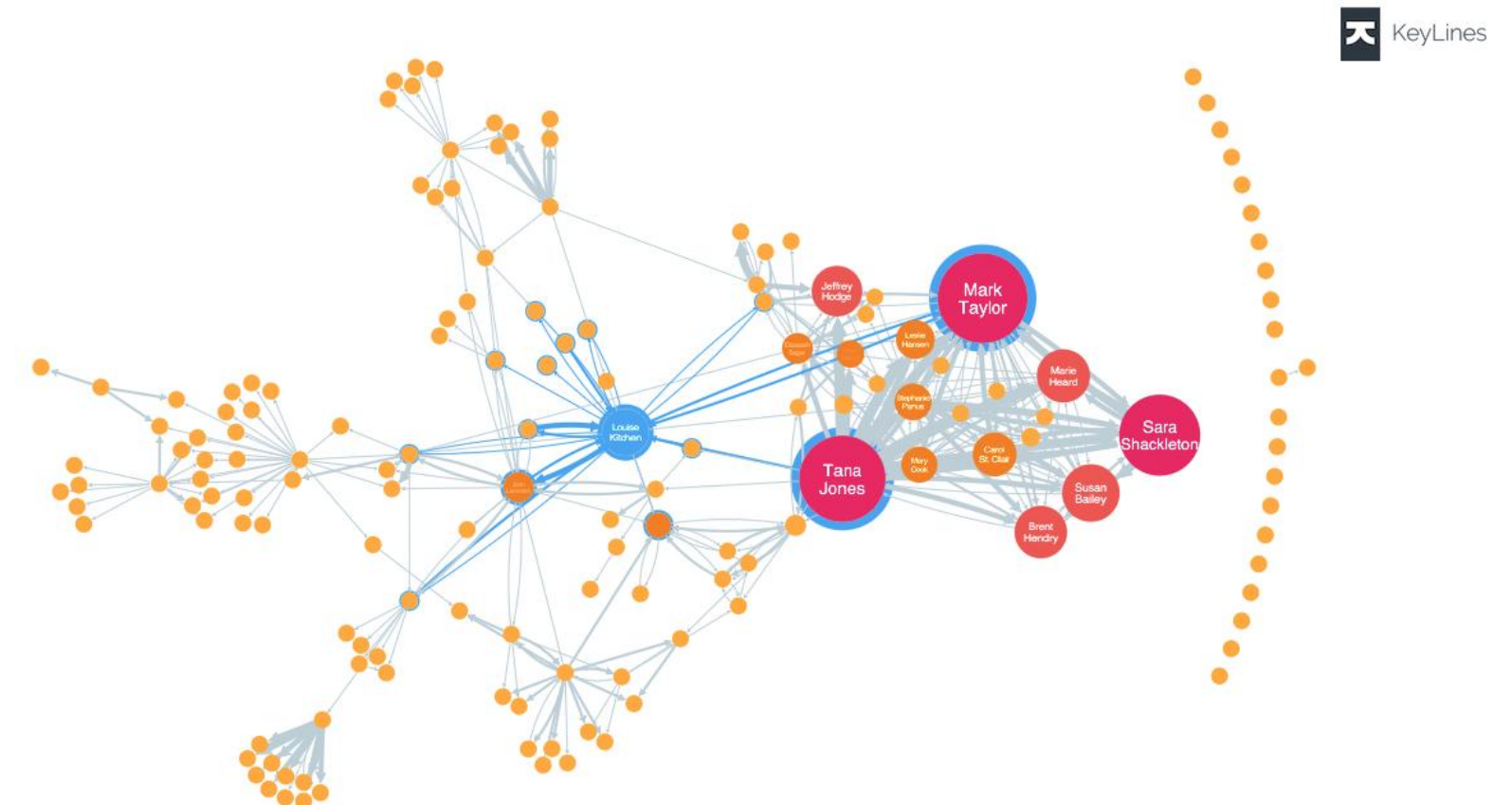
Funcionamiento: Utilizan un modelo de grafos, donde los nodos representan entidades (personas, lugares, objetos) y las aristas representan relaciones entre ellas. Esto permite realizar consultas complejas sobre las relaciones de los datos de manera eficiente.

Casos de uso:

- **Redes sociales:** Donde las relaciones entre personas (amistades, seguidores, etc.) son fundamentales.
- **Recomendaciones:** Como en los sistemas de recomendación de productos o amigos, que necesitan encontrar patrones en las relaciones.
- **Mapas y rutas:** Para aplicaciones que necesitan realizar cálculos complejos de rutas o conectividad.

Ejemplos:

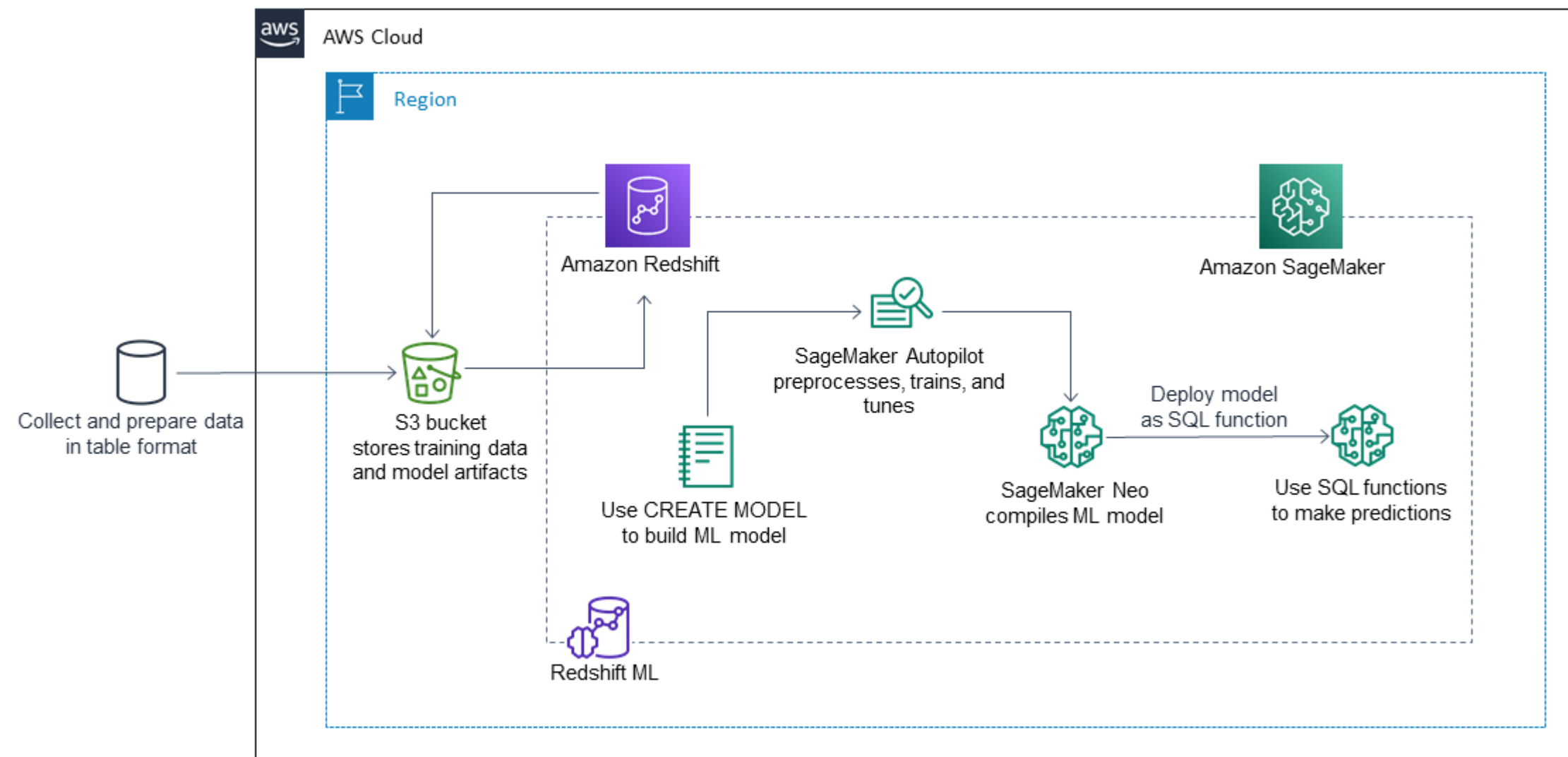
- **Neo4j:** Es una de las bases de datos de grafos más utilizadas, popular por su capacidad para realizar consultas sobre redes complejas.
- **Amazon Neptune:** Un servicio de base de datos de grafos completamente gestionado.



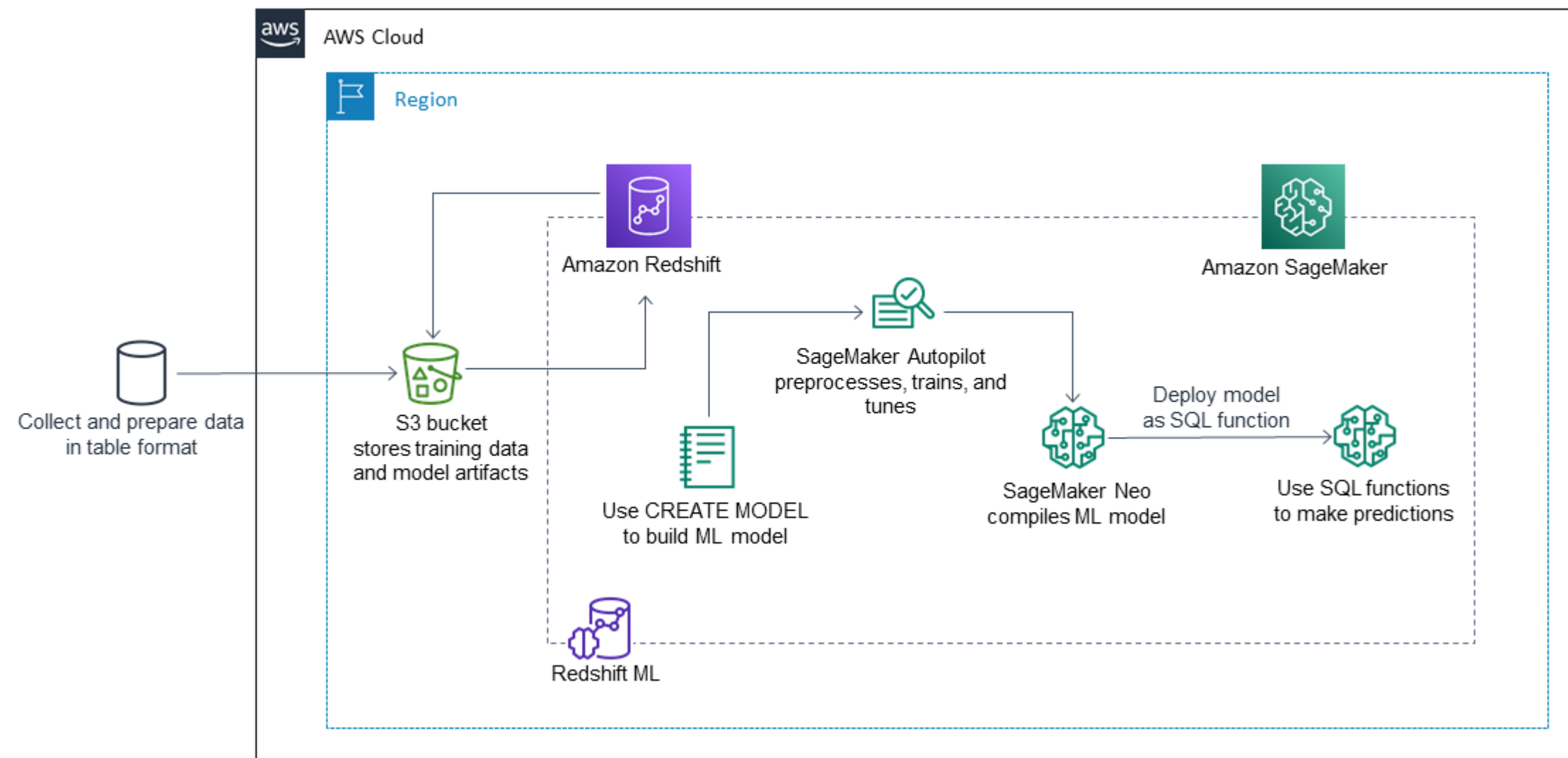
KeyLines

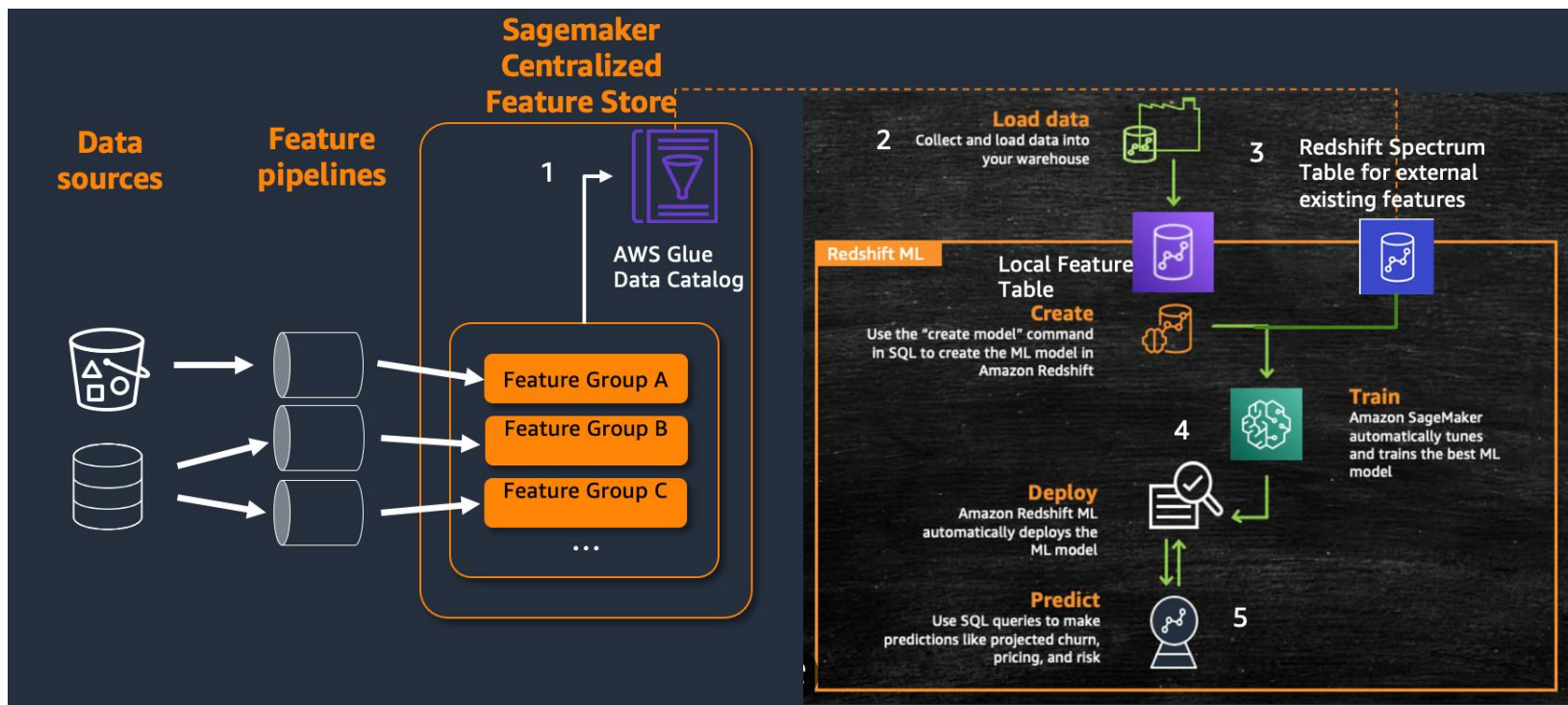
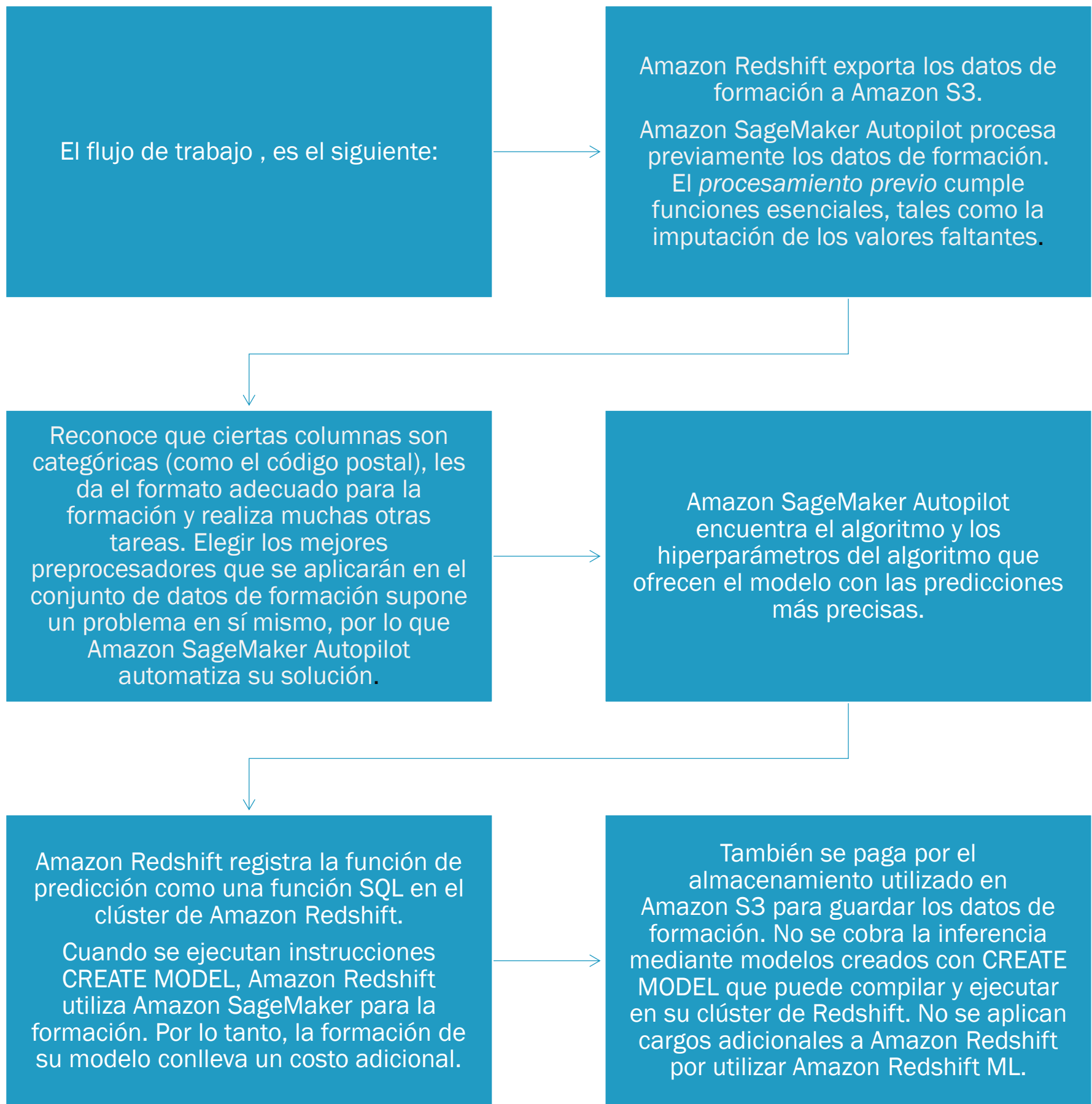
4/ Redshift

Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.



Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.





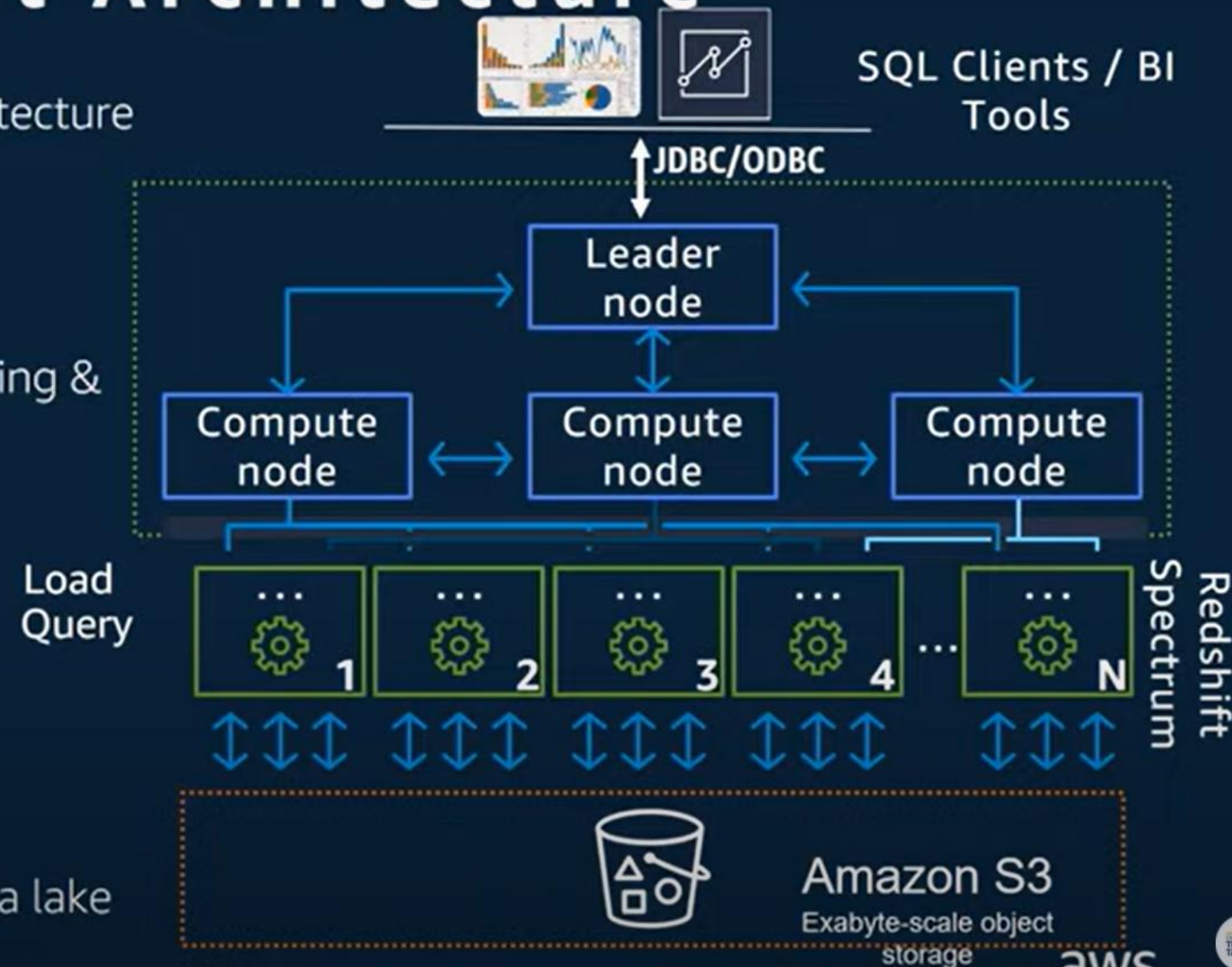
Amazon Redshift Architecture

Getting Started with Amazon Redshift - AWS Online Tech Talks

Amazon Redshift Architecture

Massively parallel, shared nothing architecture

- Leader node
 - SQL endpoint
 - Stores metadata
 - Coordinates parallel SQL processing & ML optimizations
- Compute nodes
 - Local, columnar storage
 - Executes queries in parallel
 - Load, unload, backup, restore from S3
- Amazon Redshift Spectrum nodes
Execute queries directly against data lake



Arquitectura del sistema de almacenamiento de datos - Amazon Redshift

https://youtu.be/_y0YHG95MI48?t=124

Minuto 2

Amazon Redshift slices

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Slices

A *slice* can be thought of like a virtual compute node

- Unit of data partitioning
- Parallel query processing

Facts about slices:

- Each compute node has either 2, 16, or 32 slices
- Table rows are distributed to slices
- A slice processes only its own data

<https://youtu.be/Hur-p3kGDTA?t=1009> min
16:49

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

▶ ▶ 17:06 / 51:05 • Best Practices: Sort Keys > Desliza hacia abajo para ver más detalles

aws



Amazon Redshift Blocks

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Presiona **Esc** para salir de pantalla completa

Terminology and Concepts: Blocks

Column data is persisted to 1 MB immutable blocks

Blocks are individually encoded with 1 of 11 encodings

A full block can contain millions of values

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Desliza hacia abajo para ver más detalles

10:47 / 51:05 • Terminology and Concepts: Blocks

aws



<https://youtu.be/Hur-p3kGDTA?t=652> minuto 11:23
[Amazon Redshift: 3 Comprehensive Aspects \(hevodata.com\)](https://hevodata.com)

Amazon Redshift Zone Maps

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Zone Maps

Goal: eliminates unnecessary I/O

In-memory block metadata

- Contains per-block min and max values
- All blocks automatically have zone maps
- Effectively prunes blocks which cannot contain data for a given query

<https://youtu.be/13ilj34nkQE>

Amazon Redshift is columnar

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Columnar

Amazon Redshift uses a columnar architecture for storing data on disk

Goal: reduce I/O for analytics queries

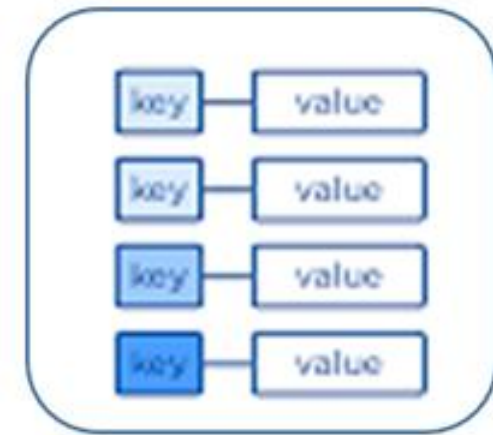
Physically store data on disk by column rather than row

Only read the column data that is required

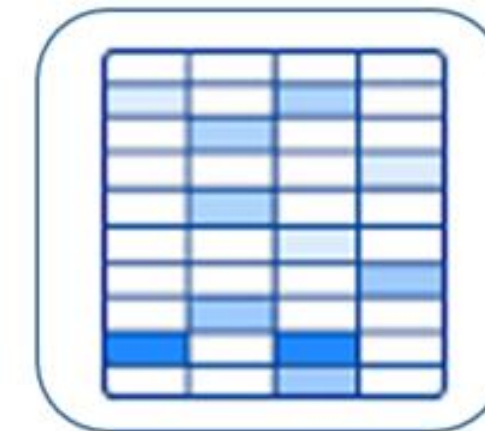
© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles



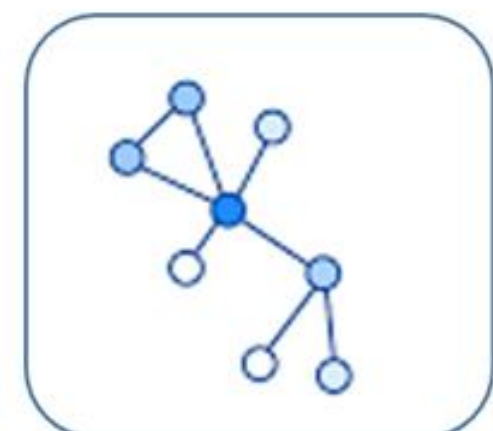
Document Store



Key-Value Store



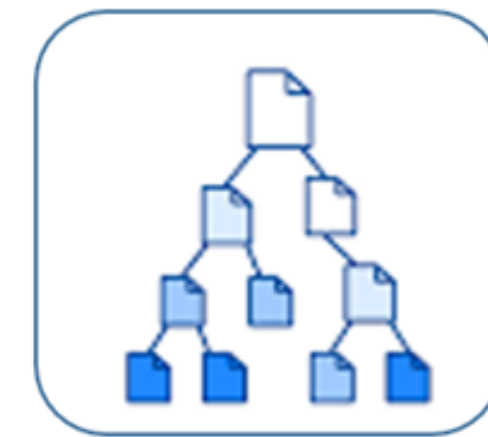
Wide-Column Store



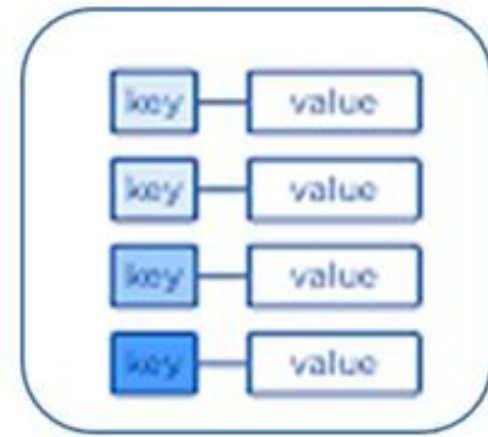
Graph Store

Amazon Redshift is columnar

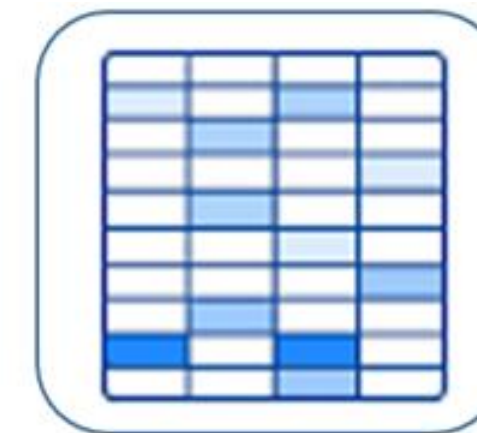
Almacenamiento en columnas - Amazon Redshift



Document Store



Key-Value Store



Wide-Column Store



Graph Store

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (  
  aid INT      --audience_id  
  ,loc CHAR(3) --location  
  ,dt DATE     --date  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

SELECT min(dt) FROM deep_dive;

Row-based storage

- Need to read everything
- Unnecessary I/O

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (  
  aid INT      --audience_id  
  ,loc CHAR(3) --location  
  ,dt DATE     --date  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

SELECT min(dt) FROM deep_dive;

Column-based storage

- Only scan blocks for relevant column

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Amazon Redshift compression

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Compression: Example

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
,loc CHAR(3)  --location
,dt DATE      --date
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

Add 1 of 11 different encodings to each column

Botón de reproducción (k)

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

aws

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Compression: Example

```
CREATE TABLE deep_dive (
  aid INT      ENCODE ZSTD
,loc CHAR(3)  ENCODE BYTEDICT
,dt DATE      ENCODE RUNLENGTH
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

More efficient compression is due to storing the same data type in the columnar architecture

Columns grow and shrink independently

Reduces storage requirements

Reduces I/O

Presiona Esc para salir de pantalla completa

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

aws

Amazon Redshift data sorting

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Data Sorting

Presiona **Esc** para salir de pantalla completa

Goal: make queries run faster by increasing the effectiveness of zone maps and reducing I/O

Impact: enables range-restricted scans to prune blocks by leveraging zone maps

Achieved with a SORTKEY defined on one or more columns

Optimal sort key is dependent on:

- Query patterns
- Business requirements
- Data profile

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Sort Key: Example

Presiona **Esc** para salir de pantalla completa

Add a sort key to one or more columns to physically sort the data on disk

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
  ,loc CHAR(3) --location
  ,dt  DATE    --date
) SORT KEY (dt, loc);
```

deep_dive		
aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

deep_dive (sorted)		
aid	loc	dt
3	SFO	2017-04-01
4	JFK	2017-05-14
2	JFK	2017-10-20
1	SFO	2017-10-20

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

<https://youtu.be/Hur-p3kGDTA?t=720>

Amazon Redshift data distribution

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Data Distribution

Distribution style is a table property which dictates how that table's data is distributed throughout the cluster:

- **KEY:** value is hashed, same value goes to same location (slice)
- **ALL:** full table data goes to the first slice of every node
- **EVEN:** round robin

Goals:

- Distribute data evenly for parallel processing
- Minimize data movement during query processing

key and what we essentially do is we take

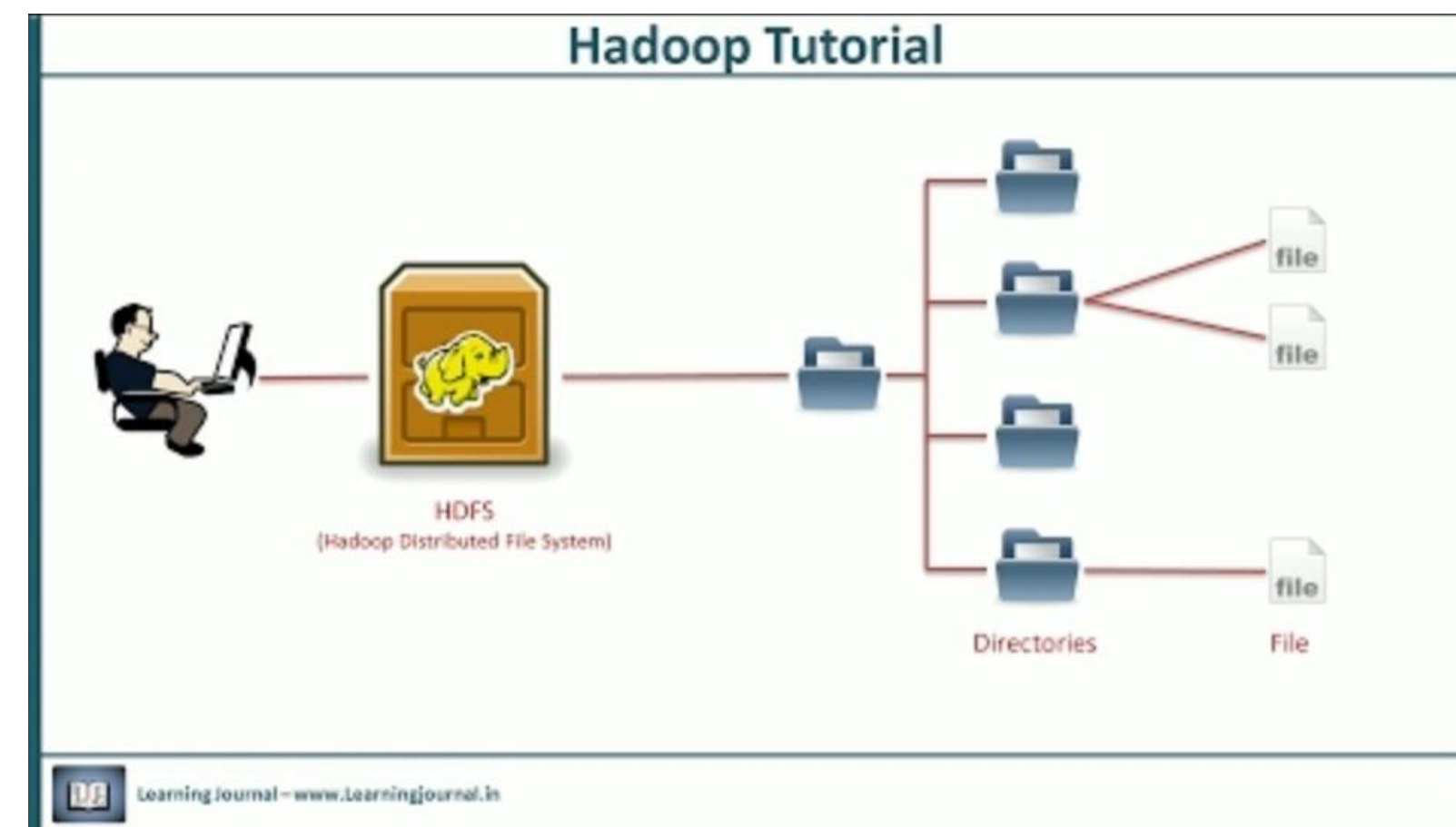
© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.
17:51 / 51:05 • Best Practices: Sort Keys

<https://youtu.be/Hur-p3kGDTA?t=1071>

1.1 ¿Qué es HDFS?

HDFS (Hadoop Distributed File System) es el sistema de archivos distribuido del proyecto **Apache Hadoop**, diseñado para almacenar y gestionar grandes volúmenes de datos (Big Data) de forma eficiente y segura en un entorno distribuido. A diferencia de los sistemas de archivos tradicionales, HDFS permite el almacenamiento de datos en un clúster de máquinas, con capacidades de escalabilidad y alta disponibilidad.

- **Replicación de datos.** HDFS hace que los datos estén siempre disponibles, aunque falle uno de los ordenadores del clúster. Gracias a este sistema se pueden replicar datos y seguir su procesamiento.
- **Tolerancia a fallos.** La capacidad de replicación y de copia del sistema de archivos distribuidos hace que el sistema sea más tolerante a posibles fallos.
- **Escalabilidad.** Al funcionar a través de un clúster de ordenadores, es mucho más sencillo escalar la capacidad de procesamiento añadiendo o quitando ordenadores.
- **Rendimiento.** Al almacenar los datos en un sistema distribuido, HDFS hace que se puedan procesar de forma paralela. Esto hace que se reduzca el tiempo de procesamiento y, por tanto, se mejore el rendimiento en el mismo.

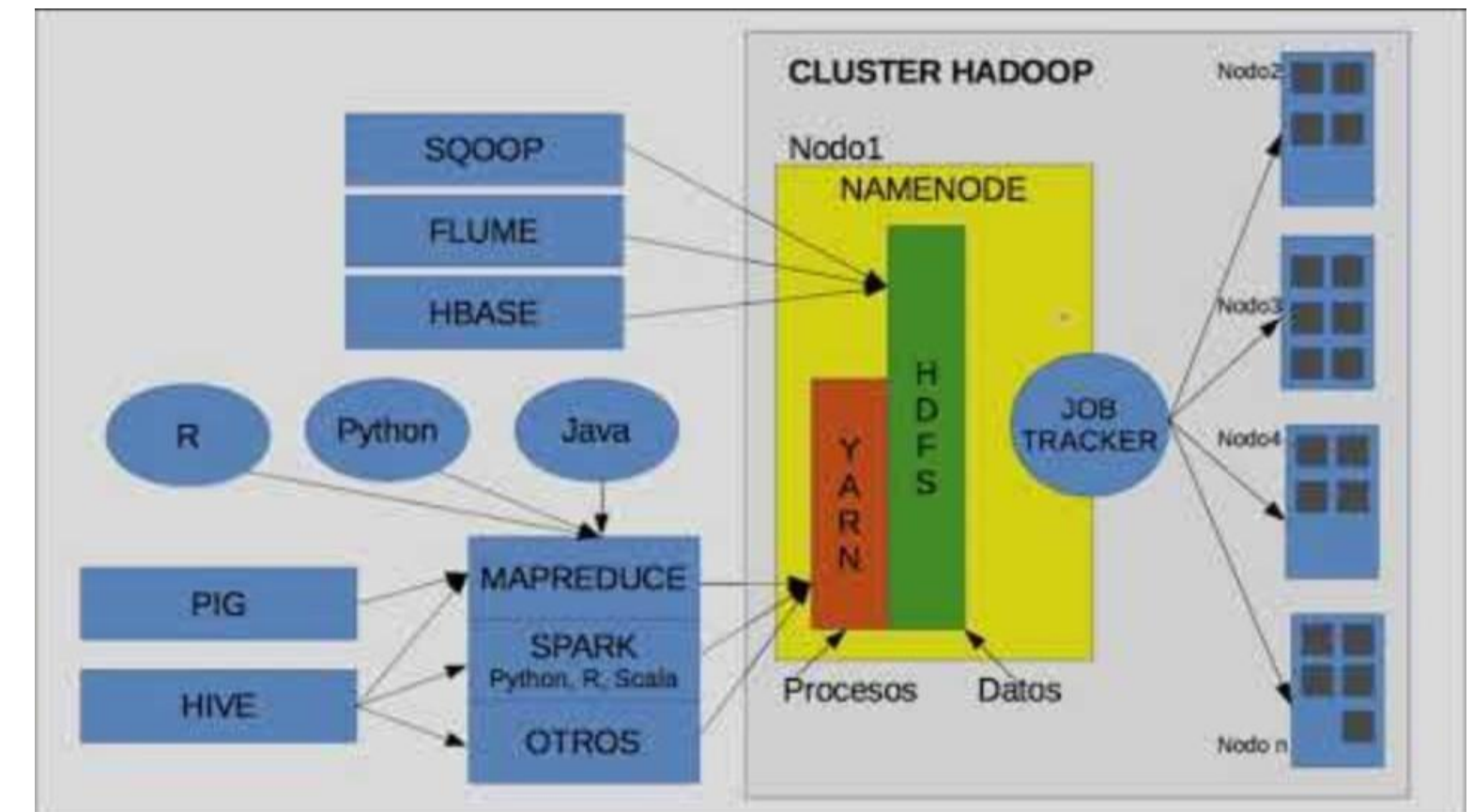


HDFS sigue un modelo maestro-esclavo, donde:

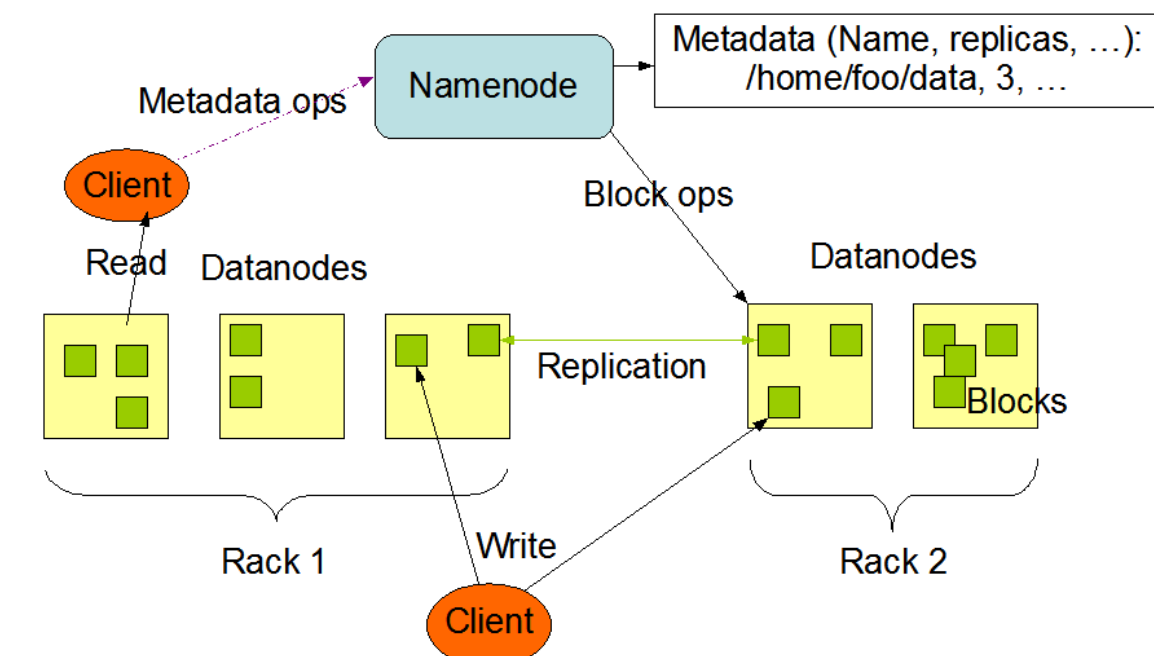
- El **NameNode** es el servidor central que administra el sistema de archivos y conoce la ubicación de los bloques de datos.
- Los **DataNodes** almacenan los datos reales en bloques y ejecutan las operaciones de lectura/escritura solicitadas por los clientes.

Características principales:

- **Distribución de datos:** Los archivos se dividen en bloques (típicamente de 128 MB) que se distribuyen a través de los DataNodes.
- **Replicación:** Cada bloque tiene múltiples copias distribuidas en varios nodos para garantizar la disponibilidad en caso de fallo.



HDFS Architecture



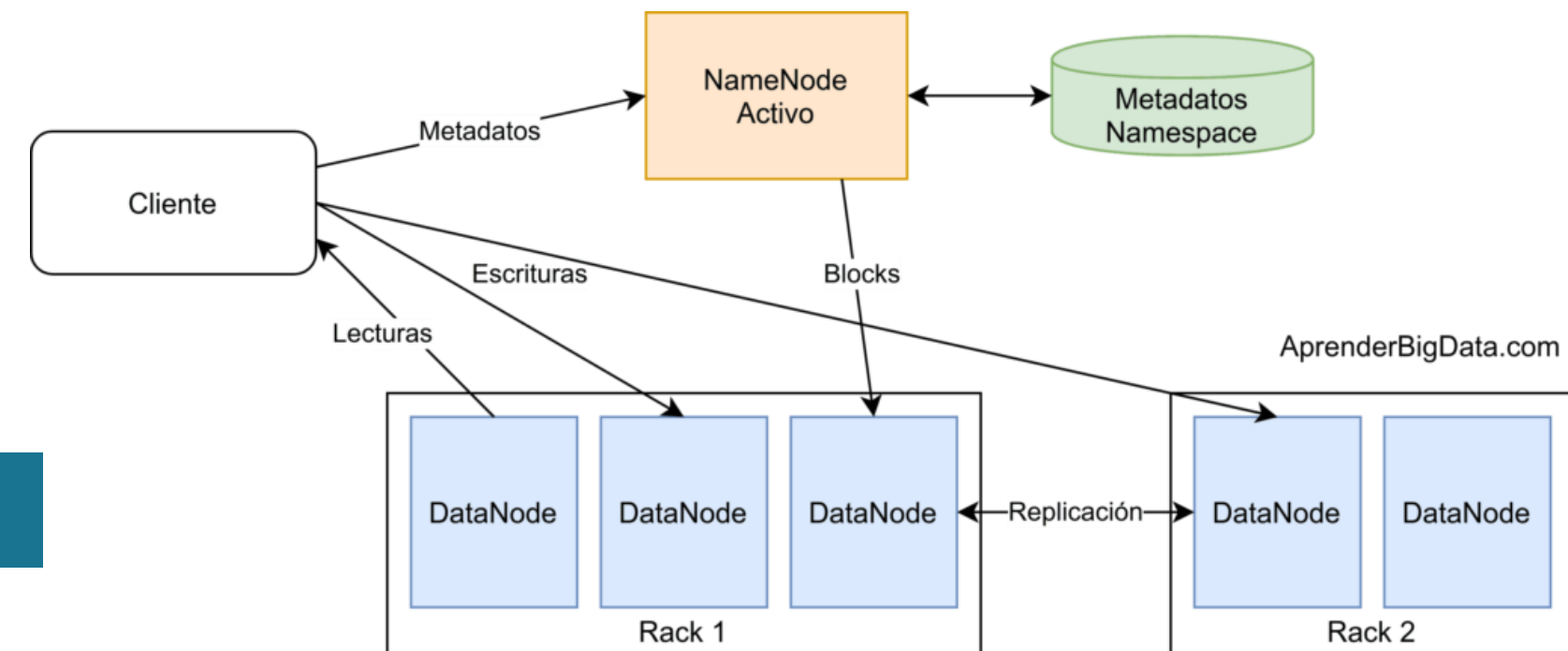
NameNode

- El NameNode es el núcleo de la arquitectura HDFS. Administra el sistema de archivos y el namespace, rastreando dónde se almacenan los bloques en el clúster. No almacena los datos reales, sino la **metadata** (información de ubicación de los bloques y estructura de archivos).
- **Funciones del NameNode:**
 - Gestionar la creación, eliminación y modificación de archivos y directorios.
 - Mantener un mapa de dónde se encuentran los bloques de datos en los DataNodes.
- **Alta disponibilidad:** Si el NameNode falla, puede ser una vulnerabilidad crítica. Por ello, se utilizan NameNodes secundarios (o standby) para tener redundancia.

DataNode

Los DataNodes son los encargados de almacenar los datos reales. Los archivos se dividen en bloques y se distribuyen entre varios DataNodes.

- **Funciones del DataNode:**
 - Almacenar y replicar bloques de datos.
 - Realizar operaciones de lectura y escritura según lo soliciten los clientes.
 - Informar periódicamente al NameNode sobre el estado de los bloques que contienen.
- **2.3 Bloques y replicación**
- HDFS divide los archivos en **bloques** de tamaño fijo (128 MB por defecto), lo que permite una distribución eficiente de los datos entre diferentes nodos del clúster.
- **Replicación:** Cada bloque se replica en varios DataNodes (típicamente tres copias por bloque) para asegurar la tolerancia a fallos. Si un nodo falla, los datos aún pueden ser accesibles desde otras réplicas.



Airbnb

Airbnb almacena y procesa grandes volúmenes de datos sobre reservas, interacciones de usuarios, precios, y ubicaciones utilizando Hadoop. Los casos de uso incluyen:

- **Análisis de datos de usuarios y reservas** para optimizar la estrategia de precios dinámicos y maximizar el ingreso para los anfitriones.
- **Modelos de machine learning** para mejorar las recomendaciones de propiedades a los usuarios, basadas en el análisis del comportamiento anterior.
- **Mejoras en la búsqueda de propiedades:** Utiliza Hadoop para analizar las consultas de búsqueda y mejorar los algoritmos que sugieren los lugares más relevantes.



Netflix

Netflix usa Hadoop para gestionar y analizar datos masivos provenientes de las interacciones de los usuarios con su plataforma. Los principales casos de uso incluyen:

- **Recomendaciones personalizadas:** Netflix utiliza Hadoop para analizar el historial de visualizaciones y patrones de comportamiento de los usuarios, mejorando sus algoritmos de recomendación de contenido.
- **Análisis predictivo:** Hadoop permite a Netflix predecir qué contenido será popular y ajustar su catálogo de películas y series en consecuencia.
- **Optimización de la infraestructura de red:** Analizando datos de streaming en tiempo real para mejorar la calidad de video y reducir tiempos de buffering.



Gracias